SWIFT: Self-guided Weak-to-strong dIFfusion models via Target extrapolation

Yanjin He[1,2,♯], **Cuifeng Shen**[2], **Wanzhou Lei**[2,3,♯], **Maohua Li**[2,4,♯]
Tao Lan[2], **Kan Liu**[2], **Hanlin Tang**[†,2], **Qing Qu**[1]

[1]University of Michigan, Ann Arbor [2]Alibaba Group
[3]University of California, Berkeley [4]Nanjing University

Abstract

Training high-quality diffusion models is computationally expensive, and inference-time guidance often adds further sampling cost. In this work, we introduce SWIFT, a simple yet effective self-guided training method that uses a frozen weak denoiser to improve a stronger model without adding inference overhead. For each noisy input, SWIFT extrapolates the clean regression target away from the weak denoiser's prediction, amplifying components that remain underfit. The weak guide can be an early training checkpoint, requires no backpropagation during main-model training, and is discarded afterward. The resulting model retains the original sampler and supports both conditional and unconditional generation without label dropout or an unconditional branch. We explain this mechanism through a spectral analysis of finite-time population gradient flow in an RKHS. With an early-stopped weak guide, we prove that a nonempty range of extrapolation strengths yields strictly lower posterior-mean error than matched standard training, despite shifting the regression target away from the posterior mean. Experiments on CIFAR-10 and ImageNet-256 demonstrate improvements across model scales and representations. In one ImageNet-256 configuration, SWIFT surpasses the baseline's best FID using roughly half the total training compute, including guide preparation and training overhead. On RAEv2, it achieves FID 1.15 without inference-time guidance and **0.99** with guidance. These results establish target extrapolation as an effective approach to weak-to-strong diffusion training with efficient single-model inference.

E-mail: yanjinhe@umich.edu, wanghaisheng.whs@alibaba-inc.com

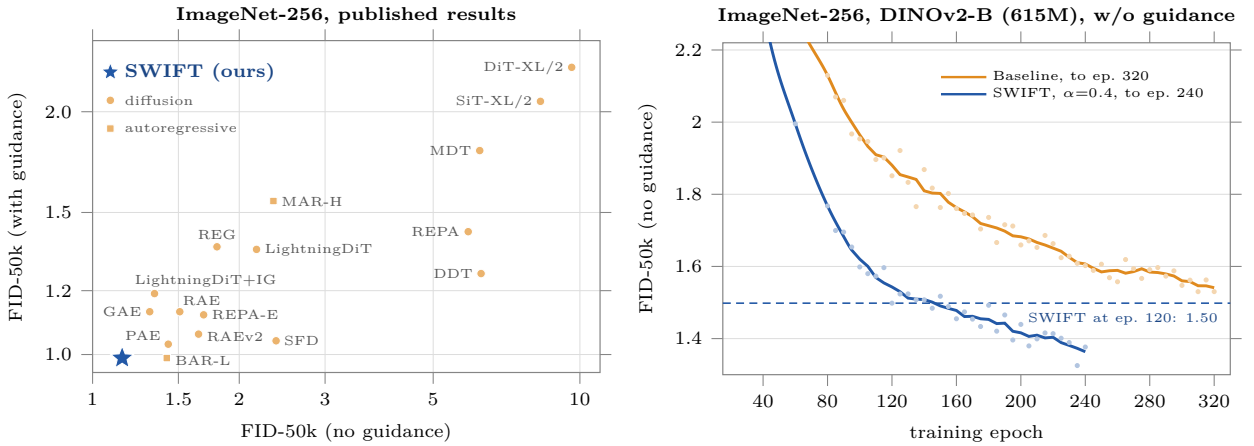

Figure 1 SWIFT **reaches sub-**1.0 **gFID on ImageNet-256.** *Left:* FID-50K without and with inference-time guidance for representative published ImageNet-256 models and SWIFT (Table 8). Our RAEv2/DINOv3-L model reaches 1.15 gFID without inference-time guidance and 0.99 with retuned internal guidance (Appendix E.3). *Right:* on the matched RAE/DINOv2-B configuration [22, 30, 38], the advantage of SWIFT persists throughout extended training (Appendix E.4). Dots denote raw evaluations and lines show five-point moving averages.

[†]Project lead [♯]Work done during internship at Alibaba

1 Introduction

Diffusion models have achieved strong generative performance across images, video, and audio [8, 9, 12]. They are typically trained with a simple denoising regression objective, yet many of the strongest generation results rely on guidance applied only at inference [7, 11]. In particular, autoguidance [11] shows that a deliberately weaker denoiser can improve a stronger one by defining an extrapolation direction during sampling. This suggests a broader role for weak models: rather than serving as teachers to imitate, they can provide directions along which a stronger predictor should move.

Autoguidance realizes this idea by extrapolating the strong model's prediction away from that of a smaller or less-trained counterpart [11]. Related training-time approaches instead use evolving model predictions, intermediate supervision, or guidance-conditioned objectives [2, 28, 36, 39] (Appendix B). These developments motivate a simple question:

> *Can weak-to-strong extrapolation be moved from inference into training, so that the final denoiser itself benefits from the correction and the weak model can be discarded afterward?*

We introduce SWIFT, a self-guided training method that uses a frozen weak denoiser to construct extrapolated regression targets. For each noisy input, instead of regressing directly toward the clean sample, SWIFT moves the target farther in the direction from the weak prediction toward and beyond the clean sample. The weak denoiser therefore acts as a *directional reference*, rather than a teacher whose output should be imitated. It can be obtained from an early checkpoint of ordinary denoising training, remains fixed during the main training run, and requires no backpropagation. After training, the guide is discarded, leaving a single denoiser and the original sampling procedure. The same construction applies to both conditional and unconditional generation.

Our motivation is finite-time underfitting: because a weak denoiser has learned less of the target function, extrapolating away from its prediction can amplify directions that remain underfit. We show theoretically that, for a nonempty range of positive extrapolation strengths, deliberately shifting the training target in this way can produce a predictor strictly closer to the original posterior mean than matched standard training (Theorem 1; proof in Appendix A.4).

Experiments on CIFAR-10 and ImageNet-256 show that the effect is not tied to a particular representation or model family. On ImageNet-256, SWIFT improves diffusion models built on conventional VAE latents, representation-autoencoder latents [26, 38], end-to-end learned latent representations, and direct pixel-space generation. Across the evaluated DiffusionBench configurations [16], the same fixed SWIFT recipe lowers unguided FID in every configuration, and the improvement persists under longer training. The complete ImageNet comparisons and training trajectories are in Appendices E.1 and E.4, and the additional CIFAR-10 experiments are in Appendix F.

SWIFT also remains complementary to inference-time guidance. When guidance is independently retuned for the baseline and SWIFT, the SWIFT model achieves lower FID in every tested configuration (Table 7 and Appendix D.2). Moreover, on RAEv2 the preferred internal-guidance scale decreases from 1.78 for the baseline to 1.20 after SWIFT training, while the best guided gFID improves from 1.04 to 0.99 (Figure 4(b); Appendix E.3). This is consistent with SWIFT internalizing part of the weak-to-strong correction during training while leaving additional room for inference-time guidance.

Our **contributions** are:

- **Weak-to-strong learning through target extrapolation.** We introduce SWIFT, which uses a frozen weak denoiser as a directional reference to extrapolate clean-sample regression targets. The guide is fixed during main-model training and discarded at inference (Section 2.2).
- **Broad improvements across diffusion representations.** We demonstrate consistent gains across CIFAR-10 and ImageNet-256, spanning VAE, RAE, end-to-end latent, and pixel-space diffusion models. The gains persist under longer training and remain after inference-time guidance is independently retuned (Sections 5.1 and 5.2).

- **A finite-training explanation.** We show theoretically that target extrapolation can strictly reduce posterior-mean error relative to matched standard training over a nonempty range of positive extrapolation strengths (Section 4).

2 SWIFT: Self-Guided Training of Diffusion Models

2.1 Preliminaries

Denoising diffusion models generate samples by reversing Gaussian corruption [8, 10]. Let $t \in [0, 1]$ index the noise level, not the training iteration, and let σ_t increase with t. We use the additive-noise parameterization

$$\boldsymbol{x}_t = \boldsymbol{x}_0 + \boldsymbol{\varepsilon}_t, \qquad \boldsymbol{x}_0 \sim p_{\text{data}}, \qquad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \sigma_t^2 \mathbf{I}_d).$$

The noise $\boldsymbol{\varepsilon}_t$ is independent of $\boldsymbol{x}_0$. We write p_t for the distribution of $\boldsymbol{x}_t$ at fixed t. When a condition $\boldsymbol{c}$ is present, it is an additional denoiser input; we suppress it below and discuss conditional and null-condition predictions in Section 3.

Training. Standard clean-sample prediction [10, 29] trains a denoiser $\mathcal{D}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t)$ to predict $\boldsymbol{x}_0$ by minimizing

$$\mathcal{L}_{\text{base}}(\boldsymbol{\theta}) = \mathbb{E}\left[\lambda(t) \left\|\mathcal{D}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) - \boldsymbol{x}_0\right\|_2^2\right], \tag{1}$$

where $t \sim p_{\text{noise}}$ is sampled independently of $\boldsymbol{x}_0 \sim p_{\text{data}}$, and p_{noise} is the distribution of noise levels used during training. The expectation also includes $\boldsymbol{\varepsilon}_t$ drawn as above, and $\lambda(t) > 0$ on the support of p_{noise}. With finite second moments, the unrestricted population minimizer of Eq. (1) is the posterior mean $\mathcal{D}^\star(\boldsymbol{x}_t, \sigma_t) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t, t]$ (Appendix A.1). For $\sigma_t > 0$, Tweedie's formula relates it to the score [4]:

$$\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t) = \frac{\mathcal{D}^\star(\boldsymbol{x}_t, \sigma_t) - \boldsymbol{x}_t}{\sigma_t^2}. \tag{2}$$

Inference. With a differentiable variance-exploding noise schedule, the probability-flow ODE is [27]

$$d\boldsymbol{x}_t = -\dot{\sigma}_t \sigma_t \nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)\, dt. \tag{3}$$

Sampling starts at a large noise level, where the noisy marginal can be approximated by a Gaussian, and integrates Eq. (3) toward smaller t. In practice, the score in Eq. (2) is formed using the trained denoiser $\mathcal{D}_{\boldsymbol{\theta}}$ in place of $\mathcal{D}^\star$. Thus the sampler depends on the denoiser reached after finite training.

2.2 Proposed Method: Learning with a Frozen Weak Guide

Building on the preliminaries, we introduce SWIFT, which trains a denoiser using extrapolated regression targets. Instead of regressing directly onto the clean sample $\boldsymbol{x}_0$ as in Eq. (1), SWIFT minimizes

$$\mathcal{L}_{\text{SWIFT}}(\boldsymbol{\theta}) = \mathbb{E}\left[\lambda(t) \left\|\mathcal{D}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) - \boldsymbol{y}_\alpha\right\|_2^2\right], \tag{4}$$

with the same expectation and weighting as in Eq. (1), but using the extrapolated target:

$$\boldsymbol{y}_\alpha = \boldsymbol{x}_0 + \alpha\big(\boldsymbol{x}_0 - \mathcal{D}_{\text{weak}}(\boldsymbol{x}_t, \sigma_t)\big) \tag{5}$$

Here, $\mathcal{D}_{\text{weak}}$ is a pretrained, frozen weak denoiser, and $\alpha \geq 0$ sets the extrapolation strength. Setting $\alpha = 0$ recovers standard denoising training. For $\alpha > 0$, the target extends beyond the clean sample in the direction away from the weak denoiser's prediction. Velocity-predicting models [18] apply the same extrapolation to the velocity, which is equivalent (Appendix D.1).

Constructing the frozen weak denoiser. We train $\mathcal{D}_{\text{weak}}$ in the ordinary way with Eq. (1) and freeze it before the main run. The default guide uses a smaller architecture; capacity and training-stage variants are studied separately. The default DiffusionBench guide has approximately one quarter as many parameters as the main denoiser and is trained for half the main model's standard training duration; the CIFAR-10 and RAEv2 guide checkpoints are specified in Appendix D.1. The method is *self-guided* because the correction comes from a weaker denoiser trained on the same task, without an external teacher, a separate guidance objective, or an auxiliary prediction head.

The intuition is to compensate for what ordinary training has not learned within a finite training budget. Extrapolating the clean target away from $\mathcal{D}_{\text{weak}}$ emphasizes directions in which the weak denoiser's prediction remains inaccurate. Whether this improves the main denoiser's approximation to the posterior mean depends on how those directions relate to its own finite-training error. For a frozen weak denoiser obtained by early stopping ordinary training, Section 4 and Theorem 1 establish conditions under which SWIFT achieves strictly lower posterior-mean error than baseline training (proofs in Appendix A; a Gaussian-mixture illustration is in Appendix C).

Training and inference. During training, only the main denoiser $\mathcal{D}_{\boldsymbol{\theta}}$ is optimized with Eq. (4). The weak denoiser $\mathcal{D}_{\text{weak}}$ is evaluated without gradients and remains fixed. Thus, for a fixed clean sample, noise realization, and noise level, the regression target does not change as the main denoiser is updated. Beyond preparing the weak denoiser, each training step requires one additional weak-denoiser forward pass (about 9% more FLOPs on DiffusionBench; Appendix D.3). After training, it is discarded; sampling uses the trained denoiser and the original sampler, with no additional inference-time model evaluation (Appendix D.2).

3 From Autoguidance to Self-Guided Training

A visual comparison. Following the fractal-like Gaussian-mixture example of Karras et al. [11], we compare how different methods reproduce a distribution with thin, branching support. In Figure 2, SWIFT has the lowest reported KL divergence and outlier rate and the highest reported $\log p$ among the seven methods. Relative to GFT, the strongest baseline on these three reported metrics, its KL decreases from 0.817 to 0.728, its outlier rate from 4.97% to 3.66%, and its $\log p$ increases from -0.40 to $+0.08$. The highlighted regions show how the generated samples follow the narrow branches.

Inference-time guidance includes classifier-free guidance (CFG) [7] and autoguidance (AG) [11]. Training-based approaches include Model Guidance (MG) [28], Segmented Guidance (SGG) [36], and Guidance-Free Training (GFT) [2]. Internal Guidance (IG) [39] trains an intermediate output for guidance at sampling time. Additional comparisons appear in Appendix B and Table 2.

3.1 From Sampling-Time Guidance to Training Targets

CFG combines conditional and unconditional predictions during sampling. AG is closer to SWIFT in its use of a weak denoiser. Let $\mathcal{D}_{\text{base}}$ be a denoiser trained with the standard objective in Eq. (1). At the same noisy input and noise level, AG evaluates

$$\mathcal{D}_{\text{AG}} = (1+\alpha)\mathcal{D}_{\text{base}} - \alpha\mathcal{D}_{\text{weak}}$$

during sampling. This requires the weak prediction whenever AG is applied. In Eq. (5), SWIFT instead places the observed clean sample $\boldsymbol{x}_0$ in the position occupied by $\mathcal{D}_{\text{base}}$ and uses the extrapolation as a *training target*. The weak denoiser is discarded after training.

Conditioned on $(\boldsymbol{x}_t, t)$, the expectation of the SWIFT target is

$$(1+\alpha)\mathcal{D}^\star(\boldsymbol{x}_t, \sigma_t) - \alpha\mathcal{D}_{\text{weak}}(\boldsymbol{x}_t, \sigma_t),$$

the expression obtained by applying AG to the ideal posterior mean (Lemma A.1). AG extrapolates a baseline denoiser already trained for a finite time, while SWIFT changes the regression target before main-model training. Under the assumptions of Section 4, this change can improve approximation to the original posterior mean; it is not a guarantee for every weak denoiser.

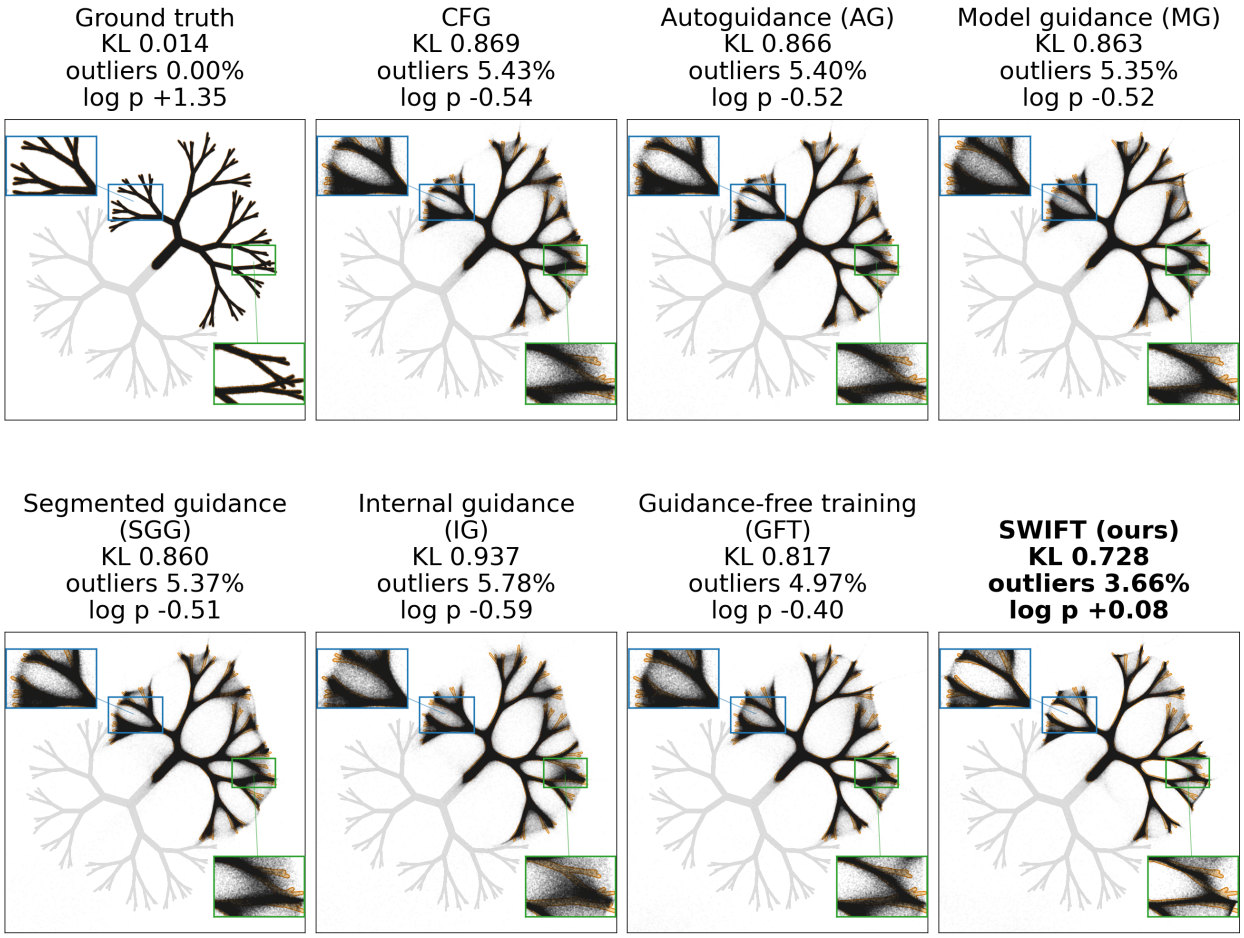

Figure 2 SWIFT **on a two-class, tree-shaped 2D toy distribution.** Ground truth and samples from CFG, AG, MG, SGG, IG, GFT, and SWIFT. Each compared method uses independently tuned settings. Blue boxes mark corresponding branching regions. Lower KL and outlier rates and higher $\log p$ are better.

3.2 Comparison with Training-Based Guidance

MG constructs a training correction using conditional and unconditional model predictions [28]. SGG studies incorporating guidance into training using an updated weak network or an intermediate prediction branch in its training-time variants [36]. GFT trains across guidance strengths using an unconditional prediction [2]. In these constructions, the auxiliary predictions can change as their models are trained. IG jointly supervises intermediate and final outputs for sampling-time guidance; it also studies training with a detached final-minus-intermediate EMA correction [39, Sec. 6].

These designs use a changing prediction in the training correction or supervise an additional output. SWIFT obtains $\mathcal{D}_{\text{weak}}$ by ordinary denoising training and freezes it before training the main denoiser. For a fixed training example and noise realization, its extrapolated target does not change as the main model is updated. This avoids joint optimization of the weak and main denoisers, which provided no clear additional endpoint benefit in our CIFAR-10 comparisons (Appendix F), and leaves a single denoiser at inference. Preparing and evaluating the weak denoiser still incur training compute, which our compute comparisons account for (Appendix D.3). Further methodological details appear in Appendix B and Eqs. (B.2) and (B.3).

4 Why SWIFT Can Help

We compare ordinary denoising and SWIFT after the same finite main-model training time. The analysis uses population gradient flow at one fixed noise level.

Assumption 1 (Population data and matched kernel training)**.** Fix a noise level t and a noisy–clean pair $(\boldsymbol{x}_0, \boldsymbol{x}_t)$ with $\mathbb{E}\|\boldsymbol{x}_0\|_2^2 < \infty$. All runs use population expectations under the same joint law. The diffusion case has $\boldsymbol{x}_t = \boldsymbol{x}_0 + \boldsymbol{\varepsilon}$, with independent $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_t^2 \mathbf{I}_d)$ and $\sigma_t > 0$; the theorem only requires the joint law and moment condition above.
Fix a bounded, measurable, symmetric positive-semidefinite scalar kernel K. Every predictor starts from zero and follows population gradient flow for normalized squared loss in the same coordinate-wise RKHS $\mathcal{H}_{\mathsf{K}}^d$. Choose $0 \le T_g < T < \infty$. An ordinary run on the clean label $\boldsymbol{x}_0$ yields the guide at time T_g; freeze it and continue the ordinary trajectory to T for the baseline. Train a separate SWIFT predictor from zero to the same time T, using the label $(1+\alpha)\boldsymbol{x}_0 - \alpha \mathcal{D}_{\mathrm{weak}}(\boldsymbol{x}_t)$ while the guide stays frozen. The two main runs thus share the data law, kernel, initialization, and optimization horizon.

Write $P_{\boldsymbol{x}_t}$ for the noisy-input distribution and $\mathcal{D}^\star(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t = \boldsymbol{x}]$ for the posterior mean. Prediction error is measured in $\mathcal{H}_2 = L^2(P_{\boldsymbol{x}_t}; \mathbb{R}^d)$. The kernel couples predictions at different noisy inputs; a finite-feature linear predictor is one example.

Objectives and training dynamics. At the fixed noise level, ordinary training and SWIFT minimize

$$\begin{aligned} \mathcal{J}_0(\boldsymbol{d}) &= \tfrac{1}{2}\mathbb{E}\|\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{x}_0\|_2^2, \\ \mathcal{J}_\alpha(\boldsymbol{d}) &= \tfrac{1}{2}\mathbb{E}\left\|\boldsymbol{d}(\boldsymbol{x}_t) - \left[(1+\alpha)\boldsymbol{x}_0 - \alpha \mathcal{D}_{\mathrm{weak}}(\boldsymbol{x}_t)\right]\right\|_2^2. \end{aligned} \tag{6}$$

For either loss, let $\boldsymbol{y}$ denote its regression label. Population gradient flow in the RKHS is

$$\frac{d\boldsymbol{d}_s}{ds} = -\nabla_{\mathcal{H}_{\mathsf{K}}^d} \tfrac{1}{2}\mathbb{E}\|\boldsymbol{d}_s(\boldsymbol{x}_t) - \boldsymbol{y}\|_2^2, \qquad \boldsymbol{d}_0 = \mathbf{0}. \tag{7}$$

Here s is optimization time, distinct from noise level t; the fixed positive loss weight $\lambda(t)$ is absorbed into s. The gradient has the explicit function form

$$\nabla \mathcal{J}(\boldsymbol{d})(\boldsymbol{x}) = \mathbb{E}[\mathsf{K}(\boldsymbol{x}, \boldsymbol{x}_t)(\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{y})]. \tag{8}$$

The corresponding derivation, including the RKHS-to-prediction-space step, is in Lemma A.2.

The resulting predictors are denoted by $\mathcal{D}_{\mathrm{weak}}$, $\mathcal{D}_{\mathrm{base}}$, and $\mathcal{D}_{\mathrm{SWIFT}}^{(\alpha)}$, respectively. The guide is weaker here through shorter training in the same RKHS; Corollary A.6 also treats a spectrally nested smaller guide.

Finite-time response. For any label $\boldsymbol{y}$, let $\boldsymbol{g}(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{y} \mid \boldsymbol{x}_t = \boldsymbol{x}]$. The two conditional-mean targets are $\boldsymbol{g}_0 = \mathcal{D}^\star$ and

$$\boldsymbol{g}_\alpha = \mathcal{D}^\star + \alpha(\mathcal{D}^\star - \mathcal{D}_{\mathrm{weak}}). \tag{9}$$

The latter identity and its denoising-MSE interpretation are proved in Lemma A.1. Define the population kernel operator on $\mathcal{H}_2$ by

$$(\boldsymbol{\mathcal{L}}\boldsymbol{f})(\boldsymbol{x}) = \int \mathsf{K}(\boldsymbol{x}, \boldsymbol{x}')\boldsymbol{f}(\boldsymbol{x}')\, dP_{\boldsymbol{x}_t}(\boldsymbol{x}'). \tag{10}$$

It is bounded, self-adjoint, and positive semidefinite. Kernel gradient flow yields

$$\frac{d\boldsymbol{d}_s}{ds} = -\boldsymbol{\mathcal{L}}(\boldsymbol{d}_s - \boldsymbol{g}), \qquad \boldsymbol{d}_s = \mathbf{H}_s \boldsymbol{g}, \quad \mathbf{H}_s := \mathbf{I} - e^{-s\boldsymbol{\mathcal{L}}}. \tag{11}$$

The response $\mathbf{H}_s$ learns a fraction $1 - e^{-s\kappa}$ of a kernel mode with eigenvalue $\kappa > 0$. The operator construction and solution of the flow appear in Appendix A.2.

Theorem 1 (Finite-time improvement from a frozen weak denoiser)**.** *Under Assumption 1,*

$$\mathcal{D}_{\text{weak}} = \mathbf{H}_{T_g}\mathcal{D}^\star, \qquad \mathcal{D}_{\text{base}} = \mathbf{H}_T\mathcal{D}^\star,$$
$$\mathcal{D}^{(\alpha)}_{\text{SWIFT}} = \mathbf{H}_T[(1+\alpha)\mathcal{D}^\star - \alpha\mathcal{D}_{\text{weak}}].$$

If $\mathbf{H}_T\mathcal{D}^\star \neq \mathbf{0}$*, the guide has strictly larger posterior-mean risk than the baseline. Moreover, for* $R(\alpha) = \|\mathcal{D}^{(\alpha)}_{\text{SWIFT}} - \mathcal{D}^\star\|^2_{\mathcal{H}_2}$*, there is an* $\alpha^\star > 0$ *such that*

$$R(\alpha) < R(0) \qquad (0 < \alpha < \alpha^\star). \tag{12}$$

Appendix A.4 proves both claims using Theorem A.3. An earlier guide has learned less of each learnable mode; a small positive extrapolation partially cancels the baseline's finite-time residual. The modewise calculation is Eq. (A.10), and a larger coefficient can overshoot. The result concerns population denoising MSE at equal main-model optimization time, with guide preparation excluded; it does not directly imply a FID improvement. A Gaussian-mixture example gives an explicit posterior mean (Lemma A.4), and for an RBF kernel Proposition A.5 verifies the nonzero-predictor condition. Appendices A.7 and A.8 give complementary finite-step and finite-sample analyses.

5 Experiments

We evaluate SWIFT on ImageNet-256 [3] and CIFAR-10 [13]. In every experiment SWIFT changes only the regression target; architectures, training schedules, samplers, and evaluation pipelines are kept the same as the baseline. On ImageNet, SWIFT uses one fixed recipe unless stated otherwise: α=0.4 with a smaller pretrained guide frozen before main-model training. The default DiffusionBench guide is trained for half the main model's standard training duration (Appendix D.1).

Evaluation setup. Inspired by DiffusionBench [16], we evaluate SWIFT across 19 ImageNet-256 model configurations. For each configuration, we compare the baseline and SWIFT with and without inference-time guidance, tuning both guidance strengths separately where checkpoints are available and identifying published baselines otherwise. We report FID [6], sFID [21], precision and recall [14], representation-space FD_r [32], and MIND [1] where available. Table 1 shows selected results; training and evaluation settings are in Appendices D.1 and D.2, with complete ImageNet-256 and CIFAR-10 results in Appendices E and F.

5.1 Generation Quality and Transfer Across Representations

Matched improvements. Table 1 isolates the target change within each model configuration. On RAE/DINOv2-B features [22, 30, 38], SWIFT lowers unguided FID from 2.09 to 1.75 and the five-feature mean FD_r from 4.13 to 3.69 after 80 epochs. The gain persists on the independent RAEv2/DINOv3-L configuration [26], where unguided FID improves from 1.56 to 1.15 within a 160-epoch training budget; the SWIFT value is selected at epoch 140 (Appendix D.2).

Transfer with one fixed recipe. Across 18 DiffusionBench configurations spanning latent and pixel representations, one fixed recipe lowers unguided FID in every case, by 16.3%–49.4% (median 31.3%). Even the three lowest-FID baselines improve by 16.3%–17.4%. FD_r and MIND each improve in at least 17 of 18 configurations in every feature space. Full results appear in Table 6.

Compatibility with inference guidance. At the best available guidance settings, SWIFT also lowers guided FID in all 18 DiffusionBench configurations (Table 7). On RAEv2, SWIFT improves internal-guidance FID from 1.04 to 0.99 (Table 1), while its unguided FID improves from 1.56 to 1.15. Representative samples from the 0.99-gFID model are shown in Figure 3; additional class-conditional samples appear in Appendix G.

Table 1 Representative ImageNet-256 results. Each pair uses the same model configuration and sampler. Guidance is tuned separately for re-evaluated pairs; source-reported guided baselines retain their published settings (Appendix D.2). $\overline{\text{FD}}_r$ and $\overline{\text{MIND}}$ are arithmetic means across Inception, ConvNeXt, DINOv2, MAE, and SigLIP features.

	Without guidance						With guidance					
Family: representation	FID↓	sFID↓	$\overline{\text{FD}}_r$ ↓	$\overline{\text{MIND}}$ ↓	Prec.↑	Rec.↑	FID↓	sFID↓	$\overline{\text{FD}}_r$ ↓	$\overline{\text{MIND}}$ ↓	Prec.↑	Rec.↑
615M parameters, 80 epochs; guidance = CFG												
RAE: DINOv2-B	2.09	4.48	4.13	21.34	0.83	0.57	1.98	4.50	4.17	20.71	0.81	0.59
+ SWIFT	**1.75**	4.26	3.69	21.54	0.81	0.61	**1.59**	4.30	3.57	19.40	0.80	0.62
VAE: SD-VAE (EMA)+reg	5.49	5.52	11.08	87.84	0.71	0.66	2.32	4.70	7.39	32.52	0.80	0.60
+ SWIFT	**3.77**	5.06	8.44	57.83	0.73	0.66	**1.81**	4.42	5.93	26.15	0.80	0.61
E2E: Qwen-VAE	4.94	5.23	9.17	95.20	0.72	0.66	1.50	4.83	4.73	24.40	0.80	0.62
+ SWIFT	**3.23**	4.89	6.47	55.63	0.73	0.66	**1.40**	4.59	3.33	23.40	0.80	0.63
Pixel: PixNerd	20.37	6.14	24.52	384.17	0.61	0.63	4.17	5.50	9.50	42.66	0.65	0.63
+ SWIFT	**10.31**	5.38	15.28	197.54	0.70	0.61	**3.38**	4.95	7.75	35.12	0.68	0.62
875M parameters, up to 160 epochs; guidance = internal guidance												
RAE: DINOv3-L	1.56	4.10	3.41	20.01	0.82	0.65	1.04	4.00	3.31	15.67	0.82	0.61
+ SWIFT	**1.15**	3.60	2.94	18.03	0.81	0.67	**0.99**	3.80	3.07	14.45	0.81	0.63

Figure 3 ImageNet-256 samples from SWIFT on RAEv2/DINOv3-L (gFID-50K 0.99). Samples are generated with the SWIFT-trained model using retuned internal guidance and a 50-step ODE sampler (Appendix D.2).

5.2 Combining SWIFT with Inference-Time Guidance

We combine SWIFT with autoguidance [11] on the DiffusionBench models and native internal REPA-head guidance on RAEv2 [26]. These mechanisms apply a weak-to-strong extrapolation at sampling time, whereas SWIFT learns a related correction during training and then discards its guide. We ask whether inference-time guidance still improves a SWIFT-trained model and whether SWIFT changes the preferred guidance strength.

(a) Guidance further improves SWIFT.

	Base	SWIFT	Guided	SWIFT+ guidance
DINOv2-B	2.09	1.75	1.70	**1.63**
SD-VAE	5.49	3.77	2.16	**1.95**
Qwen-VAE	4.94	3.23	1.79	**1.62**
PixNerd	20.37	10.31	4.02	**3.77**
RAEv2	1.56	1.15	1.04	**0.99**

(b) SWIFT shifts the guidance optimum left.

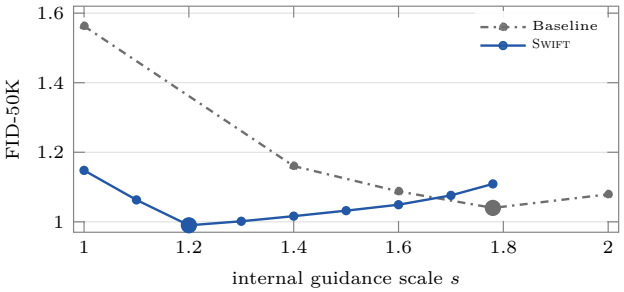

Figure 4 SWIFT **remains beneficial with inference-time guidance.** **(a)** FID with autoguidance on the DiffusionBench models and native internal REPA-head guidance on RAEv2. **(b)** On RAEv2, SWIFT shifts the preferred internal-guidance scale from 1.78 to 1.20.

Across all five configurations in Figure 4(a), the same ordering appears. SWIFT improves the unguided baseline, inference-time guidance improves it further, and independently retuned guidance on top of SWIFT gives the best FID in every case. This shows that the improvement learned during training is not replaced by inference-time guidance: even after the baseline has been separately retuned for guidance, the SWIFT model retains an additional gain.

SWIFT also changes the preferred inference-time guidance strength. On RAEv2, the optimal internal-guidance scale shifts from 1.78 for the baseline to 1.20 for SWIFT, while the best guided FID improves from 1.04 to 0.99 (Figure 4(b); Appendix E.3). The lower preferred scale is consistent with overlap between the training-time and inference-time corrections. The remaining gain shows that inference-time guidance remains complementary to the correction learned during training.

5.3 Longer Training and Training Cost

The gain persists with longer training. We continue the matched DINOv2-B pair to 240 epochs. SWIFT is ahead at all 36 evaluations from epochs 20 to 240, with the gap growing from 0.12 to 0.26 FID, with the training-time evaluator giving $1.63 \rightarrow 1.38$ at epoch 240. A separate baseline continued to 320 epochs reaches at best 1.53, still above the 1.50 reached by SWIFT at epoch 120. The matched training-time FIDs are listed in Table 9 and plotted in Figure 10; four additional trajectories appear in Figure 9 and Appendix E.4.

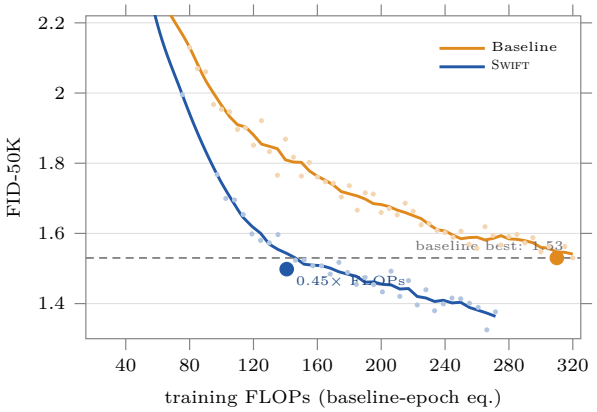

Figure 5 Training-compute comparison on DINOv2-B. SWIFT surpasses the best baseline FID observed through 320 epochs using approximately 0.45× the FLOPs needed to reach that baseline checkpoint.

Training cost. SWIFT adds 9% FLOPs per epoch for the frozen guide, plus roughly 10 baseline-equivalent epochs to pretrain it (Appendix D.3). For context, MG and SGG incur about 1.23× and 1.22× wall-clock time per training iteration in the unified SiT-B/2 comparison of Yuan et al. [36, Table 6], while GFT reports 10–20% additional computation per training update [2]. IG reports negligible extra sampling FLOPs [39, App. B]. Charging both of our costs, SWIFT at epoch 120 already surpasses the best DINOv2-B baseline result observed through 320 epochs (1.53), using approximately 0.45× the FLOPs needed to reach that baseline checkpoint (Figure 5). On RAEv2, epoch-50 SWIFT similarly improves on the epoch-160 baseline (1.34 vs. 1.56) at 0.44× the FLOPs (including guide preparation; Appendix D.3).

These time-to-quality ratios are conservative: within the observed training horizon, the baselines never reach the best SWIFT FID. No baseline time-to-quality is observed for those targets.

5.4 How to Choose α and the Weak Guide

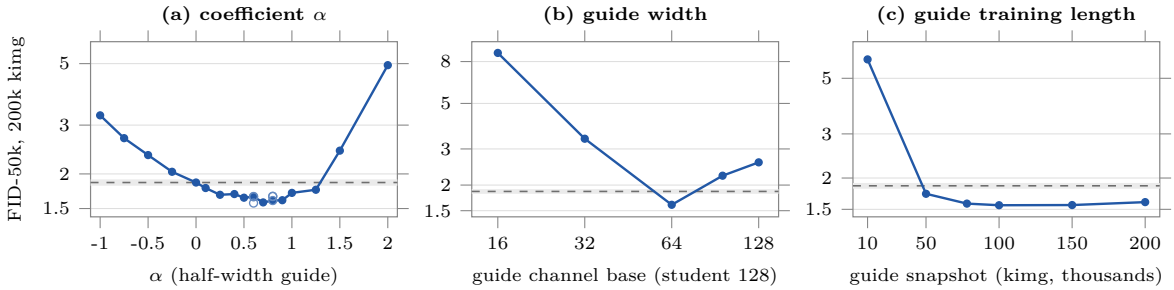

Figure 6 Choosing α and the weak guide on CIFAR-10. Moderate positive α, a half-width guide, and a sufficiently trained weak guide give the strongest results. The CIFAR-10 training and evaluation settings are in Appendices D.1 and D.2.

Choosing α. Performance is broadly U-shaped in α (Figure 6(a)). On CIFAR-10, moderate positive values form a broad favorable region around $\alpha = 0.7$–0.9, where differences are comparable to replicate noise. ImageNet shows the same qualitative behavior: on DINOv2-B, $\alpha \in [0.5, 1.0]$ substantially improves over the baseline. The precise optimum depends on the model, however, so our breadth experiments use the fixed default $\alpha = 0.4$ without tuning; in practice, a small sweep over moderate positive values is sufficient. Negative α can accelerate the initial decrease in FID, but the tested negative warm starts provide no consistent final-quality advantage (Appendix F).

Choosing the weak guide. The guide should be weaker than the student, but still competent. At $\alpha = 0.8$ on CIFAR-10, reducing the channel base from the student's 128 to 64 improves FID from 2.58 to 1.60, while substantially smaller guides degrade rapidly (Figure 6(b)). The half-width guide is also the best width across the tested α values on CIFAR-10 and consistently outperforms wider alternatives on ImageNet. Guide training length shows a similar pattern: once the guide crosses a competence threshold, performance is relatively insensitive over a broad range (Figure 6(c)). On ImageNet, freezing the guide earlier can even help, with an earlier guide outperforming the default guide in our 50k re-evaluation; the corresponding training trajectories are discussed in Appendix E.4.

A simple fixed recipe gives substantial gains. The full CIFAR-10 campaign also tests jointly trained guides, capacity switches, and coefficients that change with training time or noise level (Appendix F and Table 10). Co-training provides no clear additional endpoint benefit in the tested comparisons. Some early capacity switches and late coefficient profiles give useful CIFAR-10 refinements, whereas other schedules lose the gain; their effects depend on the guide and coefficient, and many small differences remain unresolved by repetitions. We therefore retain a frozen half-width guide and a fixed, moderate positive α as the simple default for the broader experiments.

6 Conclusion

The regression target has received comparatively little attention as a design choice in diffusion training, yet it turns out to be a simple and effective place to intervene. By carrying the clean target past itself and away from a frozen weak model, SWIFT moves a weak-to-strong correction from inference into training, leaving a single denoiser and the original sampler at deployment with only about 9% additional main-training FLOPs. Crucially, the modified target is not itself a better estimator of the posterior mean: it is deliberately biased, yet finite training on it can produce a better final predictor and better samples. This suggests a broader view of the regression target as a design interface rather than a fixed quantity—and raises the question of what other useful corrections can be written into it.

Reproducibility statement

SWIFT changes only the regression target: the objective is Eqs. (4) and (5), the frozen weak denoiser is constructed as described in Section 2.2, and everything else is the training loop of the respective framework. Appendix D.1 specifies guide architectures, freezing points, coefficients, training budgets, and hardware for the EDM, DiffusionBench, and RAEv2 models, and explains the velocity-target implementation. Appendix D.2 specifies samplers, guidance and checkpoint selection, and evaluation metrics, distinguishing re-evaluated pairs from published baselines; Appendix D.3 reports the GPU hours and the FLOPs accounting behind Figures 1 and 5. Appendix E gives all 19 ImageNet-256 configurations with and without guidance on all 15 metrics, and Appendix F the CIFAR-10 experiments, including an account of all 156 launched (152 scored) arms (Appendix F). The matched Gaussian-mixture comparison is specified in Appendix C, and all assumptions and proofs are in Appendix A. Our implementation will be made publicly available upon publication.

AI use statement

We used large language models (LLMs) in preparing this paper. They edited the text for clarity; drafted parts of the text, including appendix sections, from our experiment logs and notes; generated LaTeX and TikZ code for tables and figures; computed summary statistics from our result files. The authors reviewed and revised all LLM-assisted content and take full responsibility for the paper.

References

[1] Quentin Berthet, Yu-Han Wu, Clement Crepy, Romuald Elie, Klaus Greff, and Michael Eli Sander. MIND: Monge Inception Distance for Generative Models Evaluation. *arXiv preprint arXiv:2605.06797*, 2026. URL https://arxiv.org/abs/2605.06797.

[2] Huayu Chen, Kai Jiang, Kaiwen Zheng, Jianfei Chen, Hang Su, and Jun Zhu. Visual Generation Without Guidance. In *International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 8275–8295. PMLR, 2025. URL https://proceedings.mlr.press/v267/chen25ac.html.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009. URL https://www.image-net.org/static_files/papers/imagenet_cvpr09.pdf.

[4] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496): 1602–1614, 2011. doi: 10.1198/jasa.2011.tm11181. URL https://doi.org/10.1198/jasa.2011.tm11181.

[5] Shanghua Gao, Pan Zhou, Ming-Ming Cheng, and Shuicheng Yan. Masked Diffusion Transformer is a Strong Image Synthesizer. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 23164–23173, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/papers/Gao_Masked_Diffusion_Transformer_is_a_Strong_Image_Synthesizer_ICCV_2023_paper.pdf.

[6] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.08500.

[7] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance. *arXiv preprint arXiv:2207.12598*, 2022. URL https://arxiv.org/abs/2207.12598.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2006.11239.

[9] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video Diffusion Models. *arXiv preprint arXiv:2204.03458*, 2022. URL https://arxiv.org/abs/2204.03458.

[10] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the Design Space of Diffusion-Based Generative Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2206.00364.

[11] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a Diffusion Model with a Bad Version of Itself. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.02507.

[12] Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2009.09761.

[13] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009. URL https://cave.cs.toronto.edu/kriz/learning-features-2009-TR.pdf.

[14] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved Precision and Recall Metric for Assessing Generative Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1904.06991.

[15] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, Zhenchang Xing, Saining Xie, and Liang Zheng. REPA-E: Unlocking VAE for End-to-End Tuning with Latent Diffusion Transformers. *arXiv preprint arXiv:2504.10483*, 2025. URL https://arxiv.org/abs/2504.10483.

[16] Xingjian Leng, Jaskirat Singh, Zhanhao Liang, Ethan Smith, Martin Bell, Aninda Saha, Yuhui Yuan, and Liang Zheng. DiffusionBench: On Holistic Evaluation of Diffusion Transformers. *arXiv preprint arXiv:2606.24888*, 2026. URL https://arxiv.org/abs/2606.24888.

[17] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive Image Generation without Vector Quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.11838.

[18] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=PqvMRDCJT9t.

[19] Hangyu Liu, Jianyong Wang, and Yutao Sun. Geometric Autoencoder for Diffusion Models. *arXiv preprint arXiv:2603.10365*, 2026. URL https://arxiv.org/abs/2603.10365.

[20] Nanye Ma, Mark Goldstein, Michael S. Albergo, Nicholas M. Boffi, Eric Vanden-Eijnden, and Saining Xie. SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers. In *European Conference on Computer Vision (ECCV)*, 2024. URL https://arxiv.org/abs/2401.08740.

[21] Charlie Nash, Jacob Menick, Sander Dieleman, and Peter Battaglia. Generating Images with Sparse Representations. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 7958–7968. PMLR, 2021. URL https://proceedings.mlr.press/v139/nash21a.html.

[22] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. URL https://arxiv.org/abs/2304.07193.

[23] Yueming Pan, Ruoyu Feng, Qi Dai, Yuqi Wang, Wenfeng Lin, Mingyu Guo, Chong Luo, and Nanning Zheng. Semantics Lead the Way: Harmonizing Semantic and Texture Modeling with Asynchronous Latent Diffusion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 43664–43674, 2026. URL https://arxiv.org/abs/2512.04926.

[24] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. URL https://arxiv.org/abs/2212.09748.

[25] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved Techniques for Training GANs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1606.03498.

[26] Jaskirat Singh, Boyang Zheng, Zongze Wu, Richard Zhang, Eli Shechtman, and Saining Xie. Improved Baselines with Representation Autoencoders. *arXiv preprint arXiv:2605.18324*, 2026. URL https://arxiv.org/abs/2605.18324.

[27] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2011.13456.

[28] Zhicong Tang, Jianmin Bao, Dong Chen, and Baining Guo. Diffusion Models without Classifier-free Guidance. *arXiv preprint arXiv:2502.12154*, 2025. URL https://arxiv.org/abs/2502.12154.

[29] Pascal Vincent. A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23(7): 1661–1674, 2011. doi: 10.1162/NECO_a_00142. URL https://doi.org/10.1162/NECO_a_00142.

[30] Shuai Wang, Zhi Tian, Weilin Huang, and Limin Wang. DDT: Decoupled Diffusion Transformer. *arXiv preprint arXiv:2504.05741*, 2025. URL https://arxiv.org/abs/2504.05741.

[31] Ge Wu, Shen Zhang, Ruijing Shi, Shanghua Gao, Zhenyuan Chen, Lei Wang, Zhaowei Chen, Hongcheng Gao, Yao Tang, Jian Yang, Ming-Ming Cheng, and Xiang Li. Representation Entanglement for Generation: Training Diffusion Transformers Is Much Easier Than You Think. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://arxiv.org/abs/2507.01467.

[32] Jiawei Yang, Zhengyang Geng, Xuan Ju, Yonglong Tian, and Yue Wang. Representation Fréchet Loss for Visual Generation. *arXiv preprint arXiv:2604.28190*, 2026. URL https://arxiv.org/abs/2604.28190.

[33] Jingfeng Yao, Bin Yang, and Xinggang Wang. Reconstruction vs. Generation: Taming Optimization Dilemma in Latent Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL https://openaccess.thecvf.com/content/CVPR2025/papers/Yao_Reconstruction_vs._Generation_Taming_Optimization_Dilemma_in_Latent_Diffusion_Models_CVPR_2025_paper.pdf.

[34] Qihang Yu, Qihao Liu, Ju He, Xinyang Zhang, Yang Liu, Liang-Chieh Chen, and Xi Chen. Autoregressive Image Generation with Masked Bit Modeling. *arXiv preprint arXiv:2602.09024*, 2026. URL https://arxiv.org/abs/2602.09024.

[35] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.06940.

[36] Liangyu Yuan, Yufei Huang, Mingkun Lei, Tong Zhao, Ruoyu Wang, Changxi Chi, Yiwei Wang, and Chi Zhang. Improving Diffusion Generalization with Weak-to-Strong Segmented Guidance. *arXiv preprint arXiv:2603.20584*, 2026. URL https://arxiv.org/abs/2603.20584.

[37] Zhengrong Yue, Taihang Hu, Mengting Chen, Haiyu Zhang, Zihao Pan, Tao Liu, Zikang Wang, Jinsong Lan, Xiaoyong Zhu, Bo Zheng, and Yali Wang. What Matters for Diffusion-Friendly Latent Manifold? Prior-Aligned Autoencoders for Latent Diffusion. *arXiv preprint arXiv:2605.07915*, 2026. URL https://arxiv.org/abs/2605.07915.

[38] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion Transformers with Representation Autoencoders. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=0u1LigJaab. arXiv:2510.11690.

[39] Xingyu Zhou, Qifan Li, Xiaobin Hu, Hai Chen, and Shuhang Gu. Guiding a Diffusion Transformer with the Internal Dynamics of Itself. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11536–11545, 2026. URL https://openaccess.thecvf.com/content/CVPR2026/html/Zhou_Guiding_a_Diffusion_Transformer_with_the_Internal_Dynamics_of_Itself_CVPR_2026_paper.html.

Appendix

Overview. Appendix A proves the results of Section 4. Appendix B extends the comparison with guidance methods in Section 3. Appendix C gives a matched Gaussian-mixture comparison in which the class-conditional posterior mean is known exactly; the separate tree-shaped example is shown in Figure 2. Appendix D gives the three models, the evaluation procedure, and the compute we used. Appendix E reports every ImageNet-256 result, Appendix F the CIFAR-10 experiments, and Appendix G additional samples.

A Proofs and Extensions of the Finite-Time Analysis

We first establish the target and training-response identities, then prove Theorem 1 through a general operator comparison. The remaining subsections give the Gaussian-mixture posterior, verify learnability for Gaussian kernels, and extend the analysis to narrower guides and finite-sample fixed-feature training.

Throughout, the noise level t and the data law are those of Assumption 1. We suppress the noise argument of $\mathcal{D}^\star$ and $\mathcal{D}_{\text{weak}}$. The space $\mathcal{H}_2$, its inner product, the kernel operator $\boldsymbol{\mathcal{L}}$, and the response $\mathbf{H}_s$ are defined in Section 4. Unsubscripted function norms and inner products refer to $\mathcal{H}_2$. Write $\boldsymbol{d}_c = \mathcal{D}_{\text{base}}$ and $\boldsymbol{d}_\alpha = \mathcal{D}^{(\alpha)}_{\text{SWIFT}}$. All regression targets and predictors are square integrable.

A.1 Target identity and the quantity being compared

Write

$$\boldsymbol{\xi} = \boldsymbol{x}_0 - \mathcal{D}^\star(\boldsymbol{x}_t), \qquad \mathbb{E}[\boldsymbol{\xi} \mid \boldsymbol{x}_t] = \mathbf{0},$$
$$\boldsymbol{y}_\alpha = (1+\alpha)\boldsymbol{x}_0 - \alpha\mathcal{D}_{\text{weak}}(\boldsymbol{x}_t).$$

Lemma A.1 (SWIFT target decomposition)**.** *For every fixed square-integrable guide $\mathcal{D}_{\text{weak}}$,*

$$\boldsymbol{y}_\alpha = \mathcal{D}^\star + \alpha(\mathcal{D}^\star - \mathcal{D}_{\text{weak}}) + (1+\alpha)\boldsymbol{\xi}, \tag{A.1}$$
$$\boldsymbol{g}_\alpha(\boldsymbol{x}_t) := \mathbb{E}[\boldsymbol{y}_\alpha \mid \boldsymbol{x}_t] = \mathcal{D}^\star(\boldsymbol{x}_t) + \alpha\big(\mathcal{D}^\star(\boldsymbol{x}_t) - \mathcal{D}_{\text{weak}}(\boldsymbol{x}_t)\big). \tag{A.2}$$

Moreover,

$$\operatorname{Cov}(\boldsymbol{y}_\alpha \mid \boldsymbol{x}_t) = (1+\alpha)^2 \operatorname{Cov}(\boldsymbol{x}_0 \mid \boldsymbol{x}_t). \tag{A.3}$$

For every square-integrable predictor $\boldsymbol{d}$,

$$\mathbb{E}\|\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{y}_\alpha\|_2^2 = \|\boldsymbol{d} - \boldsymbol{g}_\alpha\|_{\mathcal{H}_2}^2 + (1+\alpha)^2\mathbb{E}\|\boldsymbol{x}_0 - \mathcal{D}^\star(\boldsymbol{x}_t)\|_2^2. \tag{A.4}$$

Consequently, the unrestricted population minimizer of the SWIFT *regression loss is $\boldsymbol{g}_\alpha$, and its posterior-mean risk is $\alpha^2\|\mathcal{D}_{\text{weak}} - \mathcal{D}^\star\|_{\mathcal{H}_2}^2$.*

Proof. Substituting $\boldsymbol{x}_0 = \mathcal{D}^\star + \boldsymbol{\xi}$ gives Eq. (A.1); conditioning on $\boldsymbol{x}_t$ gives Eq. (A.2). Since $\mathcal{D}_{\text{weak}}(\boldsymbol{x}_t)$ is fixed after conditioning on $\boldsymbol{x}_t$, the same decomposition gives Eq. (A.3). Moreover, $\boldsymbol{y}_\alpha - \boldsymbol{g}_\alpha = (1+\alpha)\boldsymbol{\xi}$ and $\boldsymbol{d} - \boldsymbol{g}_\alpha$ is $\boldsymbol{x}_t$-measurable, so

$$\mathbb{E}\big[(\boldsymbol{d} - \boldsymbol{g}_\alpha)^\top \boldsymbol{\xi}\big] = \mathbb{E}\big[(\boldsymbol{d} - \boldsymbol{g}_\alpha)^\top \mathbb{E}[\boldsymbol{\xi} \mid \boldsymbol{x}_t]\big] = 0.$$

Expanding the square proves Eq. (A.4); the remaining claims follow immediately. □

The posterior-mean risk used in Theorem 1 is also exactly the reducible part of clean-sample denoising error: for every $\boldsymbol{d}$,

$$\mathbb{E}\|\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{x}_0\|_2^2 = \|\boldsymbol{d} - \mathcal{D}^\star\|_{\mathcal{H}_2}^2 + \mathbb{E}\|\boldsymbol{x}_0 - \mathcal{D}^\star(\boldsymbol{x}_t)\|_2^2.$$

Thus posterior-mean risk and population clean-target MSE induce the same ordering of predictors. The target shift is beneficial only through the training response; it does not improve the unrestricted population optimum.

A.2 Kernel training response

The following lemma establishes the prediction-space equation and the operator properties needed for the spectral proof, without requiring finite-dimensional features or $\boldsymbol{g}$ to belong to the RKHS.

Lemma A.2 (Prediction-space kernel flow)**.** *Let $\boldsymbol{y}$ be a square-integrable vector regression target with conditional mean $\boldsymbol{g}(\boldsymbol{x}_t) = \mathbb{E}[\boldsymbol{y} \mid \boldsymbol{x}_t]$. Population gradient flow for the objective $\frac{1}{2}\mathbb{E}\|\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{y}\|_2^2$ in $\mathcal{H}_{\mathsf{K}}^d$ from the zero predictor satisfies, in prediction space,*

$$\frac{d}{ds}\boldsymbol{d}_s = -\boldsymbol{\mathcal{L}}(\boldsymbol{d}_s - \boldsymbol{g}), \qquad \boldsymbol{d}_0 = \mathbf{0}, \tag{A.5}$$

and therefore

$$\boldsymbol{d}_s = \mathbf{H}_s\boldsymbol{g}, \qquad \mathbf{H}_s = \mathbf{I} - e^{-s\boldsymbol{\mathcal{L}}}.$$

The conditional variance of $\boldsymbol{y}$ affects the value of the loss but not this population trajectory.

Proof. Conditioning on $\boldsymbol{x}_t$ gives the orthogonal decomposition

$$\mathbb{E}\|\boldsymbol{d}(\boldsymbol{x}_t) - \boldsymbol{y}\|_2^2 = \|\boldsymbol{d} - \boldsymbol{g}\|_{\mathcal{H}_2}^2 + \mathbb{E}\|\boldsymbol{y} - \boldsymbol{g}(\boldsymbol{x}_t)\|_2^2. \tag{A.6}$$

After multiplying Eq. (A.6) by $1/2$, the second term is independent of $\boldsymbol{d}$. If $\mathbf{J} : \mathcal{H}_{\mathsf{K}}^d \to \mathcal{H}_2$ sends an RKHS function to its L^2 prediction class, then $\mathbf{J}$ is bounded because $\|\mathbf{J}\boldsymbol{f}\|_{\mathcal{H}_2}^2 \le \sup_{\boldsymbol{x}} \mathsf{K}(\boldsymbol{x},\boldsymbol{x})\|\boldsymbol{f}\|_{\mathcal{H}_{\mathsf{K}}^d}^2$. The adjoint $\mathbf{J}^* : \mathcal{H}_2 \to \mathcal{H}_{\mathsf{K}}^d$ satisfies $\langle \mathbf{J}^*\boldsymbol{v}, \boldsymbol{u}\rangle_{\mathcal{H}_{\mathsf{K}}^d} = \langle \boldsymbol{v}, \mathbf{J}\boldsymbol{u}\rangle_{\mathcal{H}_2}$. Testing this identity against a kernel section in coordinate i gives

$$(\mathbf{J}^*\boldsymbol{v})_i(\boldsymbol{x}) = \int \mathsf{K}(\boldsymbol{x},\boldsymbol{x}')v_i(\boldsymbol{x}')\, dP_{\boldsymbol{x}_t}(\boldsymbol{x}').$$

Thus $\boldsymbol{\mathcal{L}} = \mathbf{J}\mathbf{J}^*$, which is bounded, self-adjoint, and positive, since $\langle \boldsymbol{v}, \boldsymbol{\mathcal{L}}\boldsymbol{v}\rangle = \|\mathbf{J}^*\boldsymbol{v}\|_{\mathcal{H}_{\mathsf{K}}^d}^2 \ge 0$. Boundedness of the kernel and the fact that $P_{\boldsymbol{x}_t}$ is a probability measure also give $\iint |\mathsf{K}(\boldsymbol{x},\boldsymbol{x}')|^2\, dP_{\boldsymbol{x}_t}(\boldsymbol{x})\, dP_{\boldsymbol{x}_t}(\boldsymbol{x}') < \infty$. The scalar integral operator is therefore Hilbert–Schmidt and compact; its finite direct sum $\boldsymbol{\mathcal{L}}$ is compact as well. For the RKHS-valued trajectory $\boldsymbol{u}_s$, the gradient is $\mathbf{J}^*(\mathbf{J}\boldsymbol{u}_s - \boldsymbol{g})$. Hence

$$\frac{d}{ds}\boldsymbol{u}_s = -\mathbf{J}^*(\mathbf{J}\boldsymbol{u}_s - \boldsymbol{g}), \qquad \boldsymbol{u}_0 = \mathbf{0}.$$

Because $\mathbf{J}^*\mathbf{J}$ is bounded, this linear Hilbert-space ODE has a unique global solution. Applying $\mathbf{J}$ to the ODE and writing $\boldsymbol{d}_s = \mathbf{J}\boldsymbol{u}_s$ proves Eq. (A.5). This argument requires only $\boldsymbol{g} \in \mathcal{H}_2$, not membership in the RKHS. Since $\boldsymbol{\mathcal{L}}$ is bounded and self-adjoint, functional calculus gives

$$\boldsymbol{d}_s = e^{-s\boldsymbol{\mathcal{L}}}\boldsymbol{d}_0 + \int_0^s e^{-(s-u)\boldsymbol{\mathcal{L}}}\boldsymbol{\mathcal{L}}\boldsymbol{g}\, du = (\mathbf{I} - e^{-s\boldsymbol{\mathcal{L}}})\boldsymbol{g}.$$

□

For finite features $\mathsf{K}(\boldsymbol{x},\boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^\top\boldsymbol{\phi}(\boldsymbol{x}')$, define $\mathbf{V} : \mathbb{R}^p \to L^2(P_{\boldsymbol{x}_t})$ by $(\mathbf{V}\boldsymbol{a})(\boldsymbol{x}) = \boldsymbol{\phi}(\boldsymbol{x})^\top\boldsymbol{a}$. Then $\mathbf{V}^*f = \mathbb{E}[\boldsymbol{\phi}(\boldsymbol{x}_t)f(\boldsymbol{x}_t)]$, $\boldsymbol{\mathcal{L}}_0 = \mathbf{V}\mathbf{V}^*$, and $\mathbf{V}^*\mathbf{V} = \mathbb{E}[\boldsymbol{\phi}(\boldsymbol{x}_t)\boldsymbol{\phi}(\boldsymbol{x}_t)^\top]$. For any eigenvalue $\kappa > 0$, $\mathbf{V}$ and $\kappa^{-1}\mathbf{V}^*$ are inverse maps between the corresponding eigenspaces. Hence the two operators have the same nonzero eigenvalues, including multiplicities.

Each $\mathbf{H}_s$ is self-adjoint with spectral multipliers in $[0,1]$; responses at different times commute because they are functions of the same operator. For the weighted diffusion loss, the normalized time is $s = 2\lambda(t)\tau$, where τ is time under the unnormalized objective. This rescaling is the same for baseline and SWIFT training.

A.3 A general commuting-response theorem

The following comparison applies to linear training responses and also yields the spectrally nested guide result in Corollary A.6.

Let $\mathbf{H}_c, \mathbf{H}_g : \mathcal{H}_2 \to \mathcal{H}_2$ denote the population responses of the main denoiser and the guide: training on a target with conditional mean $\boldsymbol{g}$ returns $\mathbf{H}_c\boldsymbol{g}$ under the main-denoiser training dynamics and $\mathbf{H}_g\boldsymbol{g}$ under the guide training dynamics. We require

$$\begin{aligned} \mathbf{H}_c = \mathbf{H}_c^*, \qquad & \mathbf{H}_g = \mathbf{H}_g^*, \\ \mathbf{0} \preceq \mathbf{H}_c \preceq \mathbf{I}, \qquad & \mathbf{0} \preceq \mathbf{H}_g \preceq \mathbf{I}, \\ \mathbf{H}_c\mathbf{H}_g = \mathbf{H}_g\mathbf{H}_c. & \end{aligned} \tag{A.7}$$

The bounds are in the quadratic-form sense. Self-adjointness and commutation give a common spectral representation, with each response multiplying target components by a factor in $[0, 1]$.

Theorem A.3 (Commuting-response improvement)**.** *Suppose Eq. (A.7) holds. Let*

$$\begin{aligned} \boldsymbol{d}_c &= \mathbf{H}_c\mathcal{D}^\star, & \mathcal{D}_{\text{weak}} &= \mathbf{H}_g\mathcal{D}^\star, \\ \boldsymbol{d}_\alpha &= \mathbf{H}_c\big((1+\alpha)\mathcal{D}^\star - \alpha\mathcal{D}_{\text{weak}}\big). \end{aligned}$$

Define $R(\alpha) = \|\boldsymbol{d}_\alpha - \mathcal{D}^\star\|^2$ *and*

$$\begin{aligned} \boldsymbol{e} &= (\mathbf{I} - \mathbf{H}_c)\mathcal{D}^\star, & \boldsymbol{q} &= \mathbf{H}_c(\mathbf{I} - \mathbf{H}_g)\mathcal{D}^\star, \\ A &= \langle \boldsymbol{e}, \boldsymbol{q} \rangle, & C &= \|\boldsymbol{q}\|^2. \end{aligned}$$

Then

$$\boldsymbol{d}_\alpha - \mathcal{D}^\star = -\boldsymbol{e} + \alpha\boldsymbol{q}, \qquad R(\alpha) = R(0) - 2A\alpha + C\alpha^2, \tag{A.8}$$

and

$$A = \langle \mathcal{D}^\star, (\mathbf{I} - \mathbf{H}_c)\mathbf{H}_c(\mathbf{I} - \mathbf{H}_g)\mathcal{D}^\star \rangle \geq 0. \tag{A.9}$$

If $A > 0$*, then* $C > 0$*, the unique minimizer is* $\alpha^\star_{\text{PM}} = A/C > 0$*, and*

$$R(\alpha) < R(0) \quad \Longleftrightarrow \quad 0 < \alpha < 2\alpha^\star_{\text{PM}}.$$

Proof. Using $\mathcal{D}_{\text{weak}} = \mathbf{H}_g\mathcal{D}^\star$ and linearity of $\mathbf{H}_c$,

$$\begin{aligned} \boldsymbol{d}_\alpha - \mathcal{D}^\star &= \mathbf{H}_c\mathcal{D}^\star + \alpha\mathbf{H}_c(\mathbf{I} - \mathbf{H}_g)\mathcal{D}^\star - \mathcal{D}^\star \\ &= -(\mathbf{I} - \mathbf{H}_c)\mathcal{D}^\star + \alpha\mathbf{H}_c(\mathbf{I} - \mathbf{H}_g)\mathcal{D}^\star \\ &= -\boldsymbol{e} + \alpha\boldsymbol{q}. \end{aligned}$$

Taking the squared norm gives Eq. (A.8). By the spectral calculus, the commuting positive operators have commuting positive square roots. Thus

$$A = \left\| (\mathbf{I} - \mathbf{H}_g)^{1/2}[\mathbf{H}_c(\mathbf{I} - \mathbf{H}_c)]^{1/2}\mathcal{D}^\star \right\|^2 \geq 0,$$

proving Eq. (A.9). When $A > 0$, $\boldsymbol{q} \neq \mathbf{0}$ and hence $C > 0$. The remaining claims follow by minimizing the strictly convex quadratic $R(0) - 2A\alpha + C\alpha^2$. □

A.4 Spectral comparison and proof of Theorem 1

Let $\boldsymbol{\mathcal{L}}_0$ be the scalar version of $\boldsymbol{\mathcal{L}}$ on $L^2(P_{\boldsymbol{x}_t})$. Compactness and positivity give orthonormal eigenfunctions u_j with positive eigenvalues κ_j. Decompose

$$\mathcal{D}^\star = \boldsymbol{d}_0^\star + \sum_j \boldsymbol{d}_j^\star u_j, \qquad \boldsymbol{d}_0^\star \in \ker \boldsymbol{\mathcal{L}}, \qquad \boldsymbol{d}_j^\star = \mathbb{E}[\mathcal{D}^\star(\boldsymbol{x}_t)u_j(\boldsymbol{x}_t)].$$

The series converges in $\mathcal{H}_2$; the list may be finite. Set $h_j = 1 - e^{-T\kappa_j}$, $h_{g,j} = 1 - e^{-T_g\kappa_j}$, and $b_j = h_j(1 - h_{g,j})$. The response lemma and target identity give the baseline coefficient $h_j \boldsymbol{d}_j^\star$ and the SWIFT coefficient $\boldsymbol{d}_{\alpha,j} = h_j[1 + \alpha(1 - h_{g,j})]\boldsymbol{d}_j^\star$. Consequently,

$$\boldsymbol{d}_{\alpha,j} - \boldsymbol{d}_j^\star = \left[\underbrace{-e^{-T\kappa_j}}_{\text{finite-time residual}} + \underbrace{\alpha b_j}_{\text{target correction}} \right] \boldsymbol{d}_j^\star. \tag{A.10}$$

All three trained predictors have zero projection onto $\ker \boldsymbol{\mathcal{L}}$, so their error in that subspace is $-\boldsymbol{d}_0^\star$ and receives no correction. These are function-space coefficients, not pointwise scaling of image pixels.

Proof of Theorem 1. Lemma A.1 and Lemma A.2 yield the three predictor expressions in the main theorem. Apply Theorem A.3 with $\mathbf{H}_c = \mathbf{H}_T$ and $\mathbf{H}_g = \mathbf{H}_{T_g}$. The responses satisfy its assumptions because they are positive contractions and commute. In the notation of that theorem, Parseval's identity gives

$$\begin{aligned} A &= \sum_j \|\boldsymbol{d}_j^\star\|_2^2 e^{-T\kappa_j} b_j, \\ C &= \sum_j \|\boldsymbol{d}_j^\star\|_2^2 b_j^2. \end{aligned} \tag{A.11}$$

Both sums converge because the spectral multipliers are bounded and $\sum_j \|\boldsymbol{d}_j^\star\|_2^2 < \infty$. The condition $\mathbf{H}_T \mathcal{D}^\star \neq \mathbf{0}$ means that at least one positive-eigenvalue coefficient is nonzero. Since $T > 0$ and $T_g < T$, $e^{-T\kappa_j} > 0$ and $b_j > 0$ for every such mode, so $A > 0$ and $C > 0$. The operator theorem therefore gives

$$R(\alpha) < R(0) \quad \Longleftrightarrow \quad 0 < \alpha < 2A/C.$$

Taking $\alpha^\star = 2A/C$ proves the improvement claim. The guide is strictly weaker because

$$\|\mathcal{D}_{\text{weak}} - \mathcal{D}^\star\|^2 - R(0) = \sum_j \|\boldsymbol{d}_j^\star\|_2^2 (e^{-2T_g\kappa_j} - e^{-2T\kappa_j}) > 0.$$

The summands are positive on every active mode, completing the proof. □

For completeness, the same expansion gives the exact modal risk

$$R(\alpha) = \|\boldsymbol{d}_0^\star\|^2 + \sum_j \|\boldsymbol{d}_j^\star\|_2^2 (-e^{-T\kappa_j} + \alpha b_j)^2. \tag{A.12}$$

Its minimizing coefficient is $\alpha_{\text{PM}}^\star = A/C$; the beneficial interval extends to twice this value. If the nonzero-predictor condition fails, both population flows remain zero. More generally, when the main-model response is an exact orthogonal projection, its residual is orthogonal to every learned correction, giving $A = 0$ and $R(\alpha) = R(0) + \alpha^2 C$. Thus restricted expressivity alone does not establish the improvement; finite-time partial fitting supplies the positive alignment.

A.5 Gaussian-mixture posterior and learnability

The main theorem applies to any square-integrable noisy–clean pair. As a concrete diffusion example, take

$$\boldsymbol{x}_0 \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \qquad \boldsymbol{x}_t = \boldsymbol{x}_0 + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_t^2 \mathbf{I}_d). \tag{A.13}$$

Here $K < \infty$, $\pi_k > 0$, $\sum_k \pi_k = 1$, $\boldsymbol{\Sigma} \succ \mathbf{0}$ is shared across components, and $\boldsymbol{\varepsilon}$ is independent of $\boldsymbol{x}_0$. The component means are arbitrary. This distribution satisfies the data and moment conditions in Assumption 1.

Let $I \in \{1, \dots, K\}$ denote the mixture component. For any component index $k \in \{1, \dots, K\}$, conditional on $I = k$,

$$\begin{aligned} \boldsymbol{x}_0 \mid I = k &\sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}), \\ \boldsymbol{x}_t \mid I = k &\sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_{\text{obs}}), \\ \boldsymbol{\Sigma}_{\text{obs}} &= \boldsymbol{\Sigma} + \sigma_t^2 \mathbf{I}_d. \end{aligned}$$

Lemma A.4 (Gaussian-mixture posterior)**.** *Under Eq. (A.13), define*

$$\begin{aligned} \ell_k(\boldsymbol{x}_t) &= \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1} \boldsymbol{x}_t - \tfrac{1}{2} \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1} \boldsymbol{\mu}_k, \\ w_k(\boldsymbol{x}_t) &= \frac{\pi_k e^{\ell_k(\boldsymbol{x}_t)}}{\sum_{j=1}^K \pi_j e^{\ell_j(\boldsymbol{x}_t)}}, \\ \overline{\boldsymbol{\mu}}(\boldsymbol{x}_t) &= \sum_{k=1}^K w_k(\boldsymbol{x}_t) \boldsymbol{\mu}_k. \end{aligned}$$

Then $w_k(\boldsymbol{x}_t) = \mathbb{P}(I = k \mid \boldsymbol{x}_t)$ *and*

$$\mathcal{D}^\star(\boldsymbol{x}_t) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t] = \mathbf{B}\boldsymbol{x}_t + (\mathbf{I}_d - \mathbf{B})\overline{\boldsymbol{\mu}}(\boldsymbol{x}_t), \qquad \mathbf{B} = \boldsymbol{\Sigma}\boldsymbol{\Sigma}_{\text{obs}}^{-1}. \tag{A.14}$$

Proof. For fixed k, $(\boldsymbol{x}_0, \boldsymbol{x}_t)$ is jointly Gaussian with $\text{Cov}(\boldsymbol{x}_0, \boldsymbol{x}_t \mid I = k) = \boldsymbol{\Sigma}$ and $\text{Cov}(\boldsymbol{x}_t \mid I = k) = \boldsymbol{\Sigma}_{\text{obs}}$. Gaussian conditioning gives

$$\begin{aligned} \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t, I = k] &= \boldsymbol{\mu}_k + \boldsymbol{\Sigma}\boldsymbol{\Sigma}_{\text{obs}}^{-1}(\boldsymbol{x}_t - \boldsymbol{\mu}_k) \\ &= \mathbf{B}\boldsymbol{x}_t + (\mathbf{I}_d - \mathbf{B})\boldsymbol{\mu}_k. \end{aligned} \tag{A.15}$$

Bayes' rule yields

$$\begin{aligned} \mathbb{P}(I = k \mid \boldsymbol{x}_t) &\propto \pi_k \exp\left[-\frac{1}{2}(\boldsymbol{x}_t - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1}(\boldsymbol{x}_t - \boldsymbol{\mu}_k)\right] \\ &\propto \pi_k \exp\left[\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1} \boldsymbol{x}_t - \frac{1}{2}\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1} \boldsymbol{\mu}_k\right], \end{aligned}$$

where the common term $-\boldsymbol{x}_t^\top \boldsymbol{\Sigma}_{\text{obs}}^{-1} \boldsymbol{x}_t / 2$ cancels. Averaging Eq. (A.15) over these weights proves Eq. (A.14). □

Learnability with a Gaussian kernel. Consider the Gaussian/RBF kernel

$$\mathsf{K}_\ell(\boldsymbol{x}_t, \boldsymbol{x}_t') = \exp\left(-\frac{\|\boldsymbol{x}_t - \boldsymbol{x}_t'\|_2^2}{2\ell^2}\right), \qquad \ell > 0. \tag{A.16}$$

Proposition A.5 (Injectivity of the Gaussian-kernel operator)**.** *For the mixture in Eq. (A.13), the marginal* $P_{\boldsymbol{x}_t}$ *has a smooth, strictly positive density on* $\mathbb{R}^d$*. The integral operator associated with Eq. (A.16) is injective on* $L^2(P_{\boldsymbol{x}_t}; \mathbb{R}^d)$*. Moreover,* $\mathcal{D}^\star \neq \mathbf{0}$*. Therefore, for every* $T > 0$*, ordinary training produces a nonzero predictor in* $\mathcal{H}_2$*, as required by the nonzero-predictor condition of Theorem 1.*

Proof. Each conditional law $\boldsymbol{x}_t \mid I = k$ is Gaussian with covariance $\boldsymbol{\Sigma}_{\text{obs}} \succ \mathbf{0}$, so its density is smooth and strictly positive; the same is true of their positive mixture.

It suffices to prove injectivity for one scalar coordinate. Let $f \in L^2(P_{\boldsymbol{x}_t})$ and suppose $\boldsymbol{\mathcal{L}}_0 f = 0$. Then $\langle f, \boldsymbol{\mathcal{L}}_0 f \rangle_{L^2(P_{\boldsymbol{x}_t})} = 0$. The signed measure $d\eta(\boldsymbol{x}_t) = f(\boldsymbol{x}_t) dP_{\boldsymbol{x}_t}(\boldsymbol{x}_t)$ has finite total variation by Cauchy–Schwarz. Use the convention $\widehat{\eta}(\boldsymbol{\omega}) = \int e^{-i\boldsymbol{\omega}^\top \boldsymbol{x}} \, d\eta(\boldsymbol{x})$. The Gaussian Fourier identity is

$$e^{-\|\boldsymbol{z}\|_2^2/(2\ell^2)} = c_{d,\ell} \int_{\mathbb{R}^d} e^{i\boldsymbol{\omega}^\top \boldsymbol{z}} e^{-\ell^2 \|\boldsymbol{\omega}\|_2^2/2} \, d\boldsymbol{\omega}, \qquad c_{d,\ell} = \left(\frac{\ell}{\sqrt{2\pi}} \right)^d.$$

The Gaussian multiplier is integrable and η has finite total variation, so Fubini's theorem applies after integrating this identity against $d\eta(\boldsymbol{x}) \, d\eta(\boldsymbol{x}')$ with $\boldsymbol{z} = \boldsymbol{x} - \boldsymbol{x}'$. It gives

$$\langle f, \boldsymbol{\mathcal{L}}_0 f \rangle = c_{d,\ell} \int_{\mathbb{R}^d} |\widehat{\eta}(\boldsymbol{\omega})|^2 e^{-\ell^2 \|\boldsymbol{\omega}\|_2^2/2} \, d\boldsymbol{\omega},$$

The Gaussian Fourier multiplier is strictly positive everywhere, so zero energy implies that $\widehat{\eta} = 0$ Lebesgue-almost everywhere. The Fourier transform of a finite signed measure is continuous, hence it vanishes everywhere; uniqueness of Fourier transforms of finite measures gives $\eta = 0$. Since $d\eta = f \, dP_{\boldsymbol{x}_t}$, $f = 0$ $P_{\boldsymbol{x}_t}$-almost surely. Applying this coordinatewise proves injectivity.

Finally, the tower property and independence of the corruption noise give

$$\mathbb{E}[\mathcal{D}^\star(\boldsymbol{x}_t) \boldsymbol{x}_t^\top] = \mathbb{E}[\boldsymbol{x}_0 \boldsymbol{x}_t^\top] = \mathbb{E}[\boldsymbol{x}_0 \boldsymbol{x}_0^\top] \succeq \boldsymbol{\Sigma} \succ \mathbf{0}.$$

Thus $\mathcal{D}^\star$ cannot vanish in $\mathcal{H}_2$. Injectivity gives $\ker \boldsymbol{\mathcal{L}} = \{\mathbf{0}\}$. For every $T > 0$, spectral calculus implies

$$\ker \mathbf{H}_T = \ker(\mathbf{I} - e^{-T\boldsymbol{\mathcal{L}}}) = \ker \boldsymbol{\mathcal{L}} = \{\mathbf{0}\}.$$

Hence $\mathcal{D}_{\text{base}} = \mathbf{H}_T \mathcal{D}^\star \neq \mathbf{0}$ in $\mathcal{H}_2$, verifying the nonzero-predictor condition of Theorem 1. □

The Gaussian kernel is injective, so every nonzero posterior mean has learnable signal. The theorem therefore does not require a deliberately underpowered function class.

A.6 Reduced-rank and spectrally nested guides

A narrower guide can be modeled by retaining a subset of the main model's learning directions. The projection below specifies the spectral nesting needed for this interpretation.

Let $\mathbf{P}_g$ be an orthogonal projection on $\mathcal{H}_2$ satisfying

$$\mathbf{P}_g \boldsymbol{\mathcal{L}} = \boldsymbol{\mathcal{L}} \mathbf{P}_g.$$

For example, $\mathbf{P}_g$ may retain any subset of the eigenfunctions of $\boldsymbol{\mathcal{L}}$; if it retains finitely many, the guide response is finite rank. For $0 < T < \infty$ and $0 \leq T_g < \infty$, define

$$\mathbf{H}_c = \mathbf{I} - e^{-T\boldsymbol{\mathcal{L}}}, \qquad \mathbf{H}_g = \mathbf{P}_g(\mathbf{I} - e^{-T_g \boldsymbol{\mathcal{L}}}). \tag{A.17}$$

Corollary A.6 (Reduced-rank guide)**.** *The operators in Eq. (A.17) satisfy Eq. (A.7). If $\mathbf{H}_c \mathcal{D}^\star \neq \mathbf{0}$, then $A > 0$ and $C > 0$ in Theorem A.3. Consequently, the* SWIFT *predictor has smaller posterior-mean risk than the baseline exactly for $0 < \alpha < 2A/C$.*

Proof. Because $\mathbf{P}_g$ is an orthogonal projection commuting with $\boldsymbol{\mathcal{L}}$, $\mathbf{H}_g$ is a positive self-adjoint contraction and commutes with $\mathbf{H}_c$. Moreover,

$$\mathbf{I} - \mathbf{H}_g - e^{-T_g \boldsymbol{\mathcal{L}}} = (\mathbf{I} - \mathbf{P}_g)(\mathbf{I} - e^{-T_g \boldsymbol{\mathcal{L}}}) \succeq \mathbf{0}.$$

All factors commute, so the alignment obeys

$$A \geq \left\langle \mathcal{D}^\star, e^{-(T+T_g)\boldsymbol{\mathcal{L}}}(\mathbf{I} - e^{-T\boldsymbol{\mathcal{L}}}) \mathcal{D}^\star \right\rangle > 0.$$

The strict inequality follows because the multiplier $e^{-(T+T_g)\kappa}(1 - e^{-T\kappa})$ is positive for every $\kappa > 0$, and $\mathbf{H}_c \mathcal{D}^\star \neq \mathbf{0}$ ensures a nonzero component in such a direction. Apply Theorem A.3. □

This corollary is the rigorous counterpart of a reduced-width guide only when the narrower model corresponds to a nested spectral subspace. Arbitrary architectural width reduction need not preserve the eigenfunctions and is not covered without an additional approximation argument.

A.7 Exact label-affine response for finite-step training

We next separate a second exact statement from the population theorem. It applies to finite data and a finite number of optimization steps, provided the features are fixed. Let $\boldsymbol{x}_{t,1}, \dots, \boldsymbol{x}_{t,n}$ be training inputs, let $\boldsymbol{\Phi} \in \mathbb{R}^{n \times p}$ be a fixed design matrix, and train the vector-valued linear predictor $\boldsymbol{\Phi}\boldsymbol{\Theta}$ with full-batch gradient descent:

$$\boldsymbol{\Theta}_{s+1} = \boldsymbol{\Theta}_s - \frac{\gamma_s}{n} \boldsymbol{\Phi}^\top (\boldsymbol{\Phi}\boldsymbol{\Theta}_s - \mathbf{Y}), \qquad s = 0, \dots, S-1,$$

where $\mathbf{Y} \in \mathbb{R}^{n \times d}$ is the label matrix. The initialization $\boldsymbol{\Theta}_0$ and step sizes γ_s are fixed independently of the labels and held the same across target choices. No convergence assumption is made.

Proposition A.7 (Finite-step label affinity)**.** *For every $S \geq 0$, there exist $\mathbf{U}_S \in \mathbb{R}^{p \times d}$ and a linear operator $\mathbf{A}_S : \mathbb{R}^{n \times d} \to \mathbb{R}^{p \times d}$, both independent of $\mathbf{Y}$, such that*

$$\boldsymbol{\Theta}_S = \mathbf{U}_S + \mathbf{A}_S(\mathbf{Y}). \tag{A.18}$$

Consequently, predictions under any fixed bounded linear evaluation map into a Hilbert prediction space are affine functions of the labels.

Proof. Set $\mathbf{Q}_s = \mathbf{I}_p - \gamma_s \boldsymbol{\Phi}^\top \boldsymbol{\Phi}/n$. Then

$$\boldsymbol{\Theta}_{s+1} = \mathbf{Q}_s \boldsymbol{\Theta}_s + \frac{\gamma_s}{n} \boldsymbol{\Phi}^\top \mathbf{Y}.$$

The assertion holds at $s = 0$ with $\mathbf{U}_0 = \boldsymbol{\Theta}_0$ and $\mathbf{A}_0 = \mathbf{0}$. If it holds at step s, it holds at step $s+1$ with

$$\mathbf{U}_{s+1} = \mathbf{Q}_s \mathbf{U}_s, \qquad \mathbf{A}_{s+1}(\mathbf{Y}) = \mathbf{Q}_s \mathbf{A}_s(\mathbf{Y}) + \frac{\gamma_s}{n} \boldsymbol{\Phi}^\top \mathbf{Y}.$$

Induction proves Eq. (A.18). Evaluation is linear in $\boldsymbol{\Theta}_S$, so it preserves affinity. □

The same proof covers mini-batch schedules fixed in advance, fixed linear preconditioners, weight decay, and momentum after augmenting the state. It does not cover label-dependent feature learning without further assumptions. In particular, the fixed-feature response has the label-affine form assumed in Proposition A.8.

A.8 Finite-sample bias–variance extension

For finite data, target extrapolation also amplifies the conditional label noise. The resulting risk depends on both residual alignment and variance inflation. The following result is exact under conditional centering, which holds in the independent-data example below but is not automatic for a guide trained on the same clean labels.

Let $\mathcal{F}$ be a sigma-algebra with respect to which the noisy training inputs, the response map, and the guide are measurable. It represents the information held fixed when averaging the remaining label randomness. Let $\mathbf{M}$ be the matrix of stacked posterior means and write the clean labels as

$$\mathbf{M} = \big(\mathcal{D}^\star(\boldsymbol{x}_{t,1}), \dots, \mathcal{D}^\star(\boldsymbol{x}_{t,n})\big)^\top \in \mathbb{R}^{n \times d}, \qquad \mathbf{X}_0 = \mathbf{M} + \mathbf{E}, \qquad \mathbb{E}[\mathbf{E} \mid \mathcal{F}] = \mathbf{0}.$$

For example, the displayed conditional-centering assumption holds for i.i.d. training pairs when the guide is trained on an independent dataset and the response map and initialization depend only on the noisy inputs and independent algorithmic randomness, with $\mathcal{F}$ generated by these noisy inputs, auxiliary data, and randomness. With $\mathbf{Y}_g$ the stacked guide predictions and $\boldsymbol{\Delta}_g = \mathbf{M} - \mathbf{Y}_g$, the SWIFT labels are

$$\mathbf{Y}_\alpha = (1+\alpha)\mathbf{X}_0 - \alpha \mathbf{Y}_g = \mathbf{M} + \alpha \boldsymbol{\Delta}_g + (1+\alpha)\mathbf{E}.$$

Let the trained predictor in a Hilbert space $\mathcal{Y}$ have the exact label-affine form $\boldsymbol{a} + \boldsymbol{\mathcal{G}}\mathbf{Y}$, where $\boldsymbol{a}$ and the bounded linear operator $\boldsymbol{\mathcal{G}}$ are $\mathcal{F}$-measurable, and $\mathcal{Y}$ is separable. Let $\boldsymbol{d}^\star_{\mathrm{ev}} \in \mathcal{Y}$ be a fixed (or $\mathcal{F}$-measurable) posterior-mean target on the evaluation distribution. Assume $\boldsymbol{a}$, $\boldsymbol{\mathcal{G}}\mathbf{M}$, $\boldsymbol{\mathcal{G}}\boldsymbol{\Delta}_g$, $\boldsymbol{d}^\star_{\mathrm{ev}}$, and $\boldsymbol{\mathcal{G}}\mathbf{E}$ are square-integrable $\mathcal{Y}$-valued random variables, and set

$$\begin{aligned} \boldsymbol{e} &= \boldsymbol{d}^\star_{\mathrm{ev}} - (\boldsymbol{a} + \boldsymbol{\mathcal{G}}\mathbf{M}), & \boldsymbol{q} &= \boldsymbol{\mathcal{G}}\boldsymbol{\Delta}_g, \\ \boldsymbol{\eta} &= \boldsymbol{\mathcal{G}}\mathbf{E}, & A &= \langle \boldsymbol{e}, \boldsymbol{q} \rangle, \\ C &= \|\boldsymbol{q}\|^2, & V &= \mathbb{E}[\|\boldsymbol{\eta}\|^2 \mid \mathcal{F}]. \end{aligned}$$

These integrability assumptions make the conditional expectations well defined and ensure $V < \infty$ almost surely. Then $\mathbb{E}[\boldsymbol{\eta} \mid \mathcal{F}] = \mathbf{0}$ and, exactly,

$$\widehat{\boldsymbol{d}}_\alpha - \boldsymbol{d}^\star_{\mathrm{ev}} = -\boldsymbol{e} + \alpha\boldsymbol{q} + (1+\alpha)\boldsymbol{\eta}.$$

Proposition A.8 (Finite-sample variance tax)**.** *Under the preceding conditional label-affine model, define*

$$R_{\mathcal{F}}(\alpha) := \mathbb{E}[\|\widehat{\boldsymbol{d}}_\alpha - \boldsymbol{d}^\star_{\mathrm{ev}}\|^2 \mid \mathcal{F}].$$

Then

$$\mathbb{E}[\|\widehat{\boldsymbol{d}}_\alpha - \boldsymbol{d}^\star_{\mathrm{ev}}\|^2 \mid \mathcal{F}] = \|\boldsymbol{e} - \alpha\boldsymbol{q}\|^2 + (1+\alpha)^2 V, \tag{A.19}$$

$$R_{\mathcal{F}}(\alpha) - R_{\mathcal{F}}(0) = -2(A - V)\alpha + (C + V)\alpha^2. \tag{A.20}$$

If $C + V > 0$, the risk-minimizing coefficient over $\alpha \geq 0$ is

$$\alpha^\star_{\mathrm{FS}} = \left[\frac{A - V}{C + V}\right]_+, \qquad [u]_+ = \max\{u, 0\}. \tag{A.21}$$

There exists a strictly improving positive coefficient if and only if $A > V$; in that case the complete improvement interval is

$$0 < \alpha < \frac{2(A - V)}{C + V},$$

and the maximal conditional risk reduction is $(A-V)^2/(C+V)$. If $C+V = 0$, then $\boldsymbol{q} = \mathbf{0}$, $\boldsymbol{\eta} = \mathbf{0}$ almost surely, and the risk is constant in α.

Proof. Since $\boldsymbol{\mathcal{G}}$ is $\mathcal{F}$-measurable, conditional centering gives $\mathbb{E}[\boldsymbol{\mathcal{G}}\mathbf{E} \mid \mathcal{F}] = \boldsymbol{\mathcal{G}}\mathbb{E}[\mathbf{E} \mid \mathcal{F}] = \mathbf{0}$. For a random operator norm this follows first on the $\mathcal{F}$-measurable events $\{\|\boldsymbol{\mathcal{G}}\| \leq m\}$, then on their union; square integrability ensures the required expectations exist. Conditionally on $\mathcal{F}$, $\boldsymbol{e}$ and $\boldsymbol{q}$ are fixed. Thus the cross term between $-\boldsymbol{e} + \alpha\boldsymbol{q}$ and $(1+\alpha)\boldsymbol{\eta}$ has conditional expectation zero, proving Eq. (A.19). Expanding and subtracting the value at $\alpha = 0$ gives Eq. (A.20). If $C + V > 0$, completing the square gives the unconstrained minimizer $(A-V)/(C+V)$ and hence Eq. (A.21). The quadratic difference has roots 0 and $2(A-V)/(C+V)$ when $A > V$, yielding the interval and maximum reduction. Finally, $C + V = 0$ implies both nonnegative terms vanish. □

Setting $V = 0$ gives the same quadratic form as Eq. (A.8), with A and C determined by the finite-sample response. With finite label noise, a beneficial coefficient exists only when residual alignment exceeds variance inflation: $A > V$. The optimizer in Eq. (A.21) is conditional on $\mathcal{F}$ and may vary with the realized inputs and guide.

A.9 Scope of the analysis

The population improvement argument requires square-integrable targets, not Gaussianity. It compares fixed-noise denoising MSE at equal main-model optimization time; guide preparation and forward passes add compute. The finite-step result assumes fixed features, and the finite-sample result additionally requires conditional centering and integrability. Neural feature learning need not satisfy these conditions, and denoising MSE alone does not guarantee improved FID or iterative sampling.

B Additional Comparisons with Diffusion Guidance Methods

Table 2 compares the six methods discussed in Section 3, including both IG formulations. We compare their guidance references, auxiliary training components, and sampling requirements.

Table 2 Training components and sampling. Checks indicate the stated property. Unconditional predictions omit the class label; extra heads are intermediate outputs with their own denoising loss. One pass means one backbone evaluation per sampling prediction, including both IG outputs. SGG is its conditional training variant. Optional backbone components and additional guidance are excluded.

Method	Guidance reference	No unconditional prediction	No extra denoising head	One forward pass per prediction
CFG [7]	unconditional prediction (same model)	×	✓	×
AG [11]	frozen weak model (same conditioning)	✓	✓	×
MG [28]	conditional and unconditional EMA predictions	×	✓	✓
SGG (train) [36]	unconditional or intermediate prediction, depending on noise level	×	×	✓
IG (main) [39]	supervised intermediate prediction	✓	×	✓
IG (training) [39]	final and intermediate EMA predictions	✓	×	✓
GFT [2]	unconditional prediction (current model)	×	✓	✓
SWIFT	**pretrained, frozen weak model (same conditioning)**	✓	✓	✓

Common notation. We write the diffusion objectives in one regression parameterization, suppressing inputs, expectations, and noise-dependent loss weights. Let $\boldsymbol{q}$ be the sample-wise label (clean sample, noise, or velocity), $\boldsymbol{f}$ the final conditional prediction, $\boldsymbol{u}$ the unconditional prediction, and $\boldsymbol{h}$ a supervised intermediate prediction. Bars denote EMA predictions and sg stops gradients. All outputs and labels in each expression use the same parameterization.

CFG and AG: extrapolation at sampling. CFG retains the ordinary denoising target and trains conditional and unconditional outputs through label dropout; sampling uses $\boldsymbol{f} + w(\boldsymbol{f} - \boldsymbol{u})$ [7, Algorithms 1–2]. AG also retains ordinary denoising losses, but samples with $\boldsymbol{f} + \alpha(\boldsymbol{f} - \boldsymbol{g})$, where $\boldsymbol{g}$ is a separately trained, less accurate model with the same conditioning [11, Sec. 4]. AG's frozen weak model is a sampling reference, not a target in main training.

MG, SGG, and IG's training variant: prediction-gap targets. Their final-output losses have the form

$$\mathcal{L}_\Delta = \|\boldsymbol{f} - [\boldsymbol{q} + \lambda\,\mathrm{sg}(\boldsymbol{\Delta})]\|_2^2, \qquad \begin{aligned} \boldsymbol{\Delta}_{\mathrm{MG}} &= \bar{\boldsymbol{f}} - \bar{\boldsymbol{u}}, \\ \boldsymbol{\Delta}_{\mathrm{SGG}} &= \begin{cases} \boldsymbol{f} - \boldsymbol{u}, & \text{high noise}, \\ \boldsymbol{f} - \boldsymbol{h}, & \text{low noise}, \end{cases} \\ \boldsymbol{\Delta}_{\text{IG-train}} &= \bar{\boldsymbol{f}} - \bar{\boldsymbol{h}}. \end{aligned} \tag{B.1}$$

MG uses conditional/unconditional EMA predictions [28, Eqs. 17–18]. SGG applies its correction within a chosen time interval, switches the weak source across noise levels, and supervises its intermediate branch

against $\boldsymbol{q}$ [36, Sec. 4.3 and App. D.1]. The same paper's training-time AG variant instead uses a separate small model, updated once per four main-model updates. Its strong-minus-weak correction therefore evolves during training. IG's training variant also supervises the intermediate output and uses the EMA gap in Eq. (B.1) [39, Sec. 6, Eq. 7]. The entire gap, including its strong prediction, is detached in these target corrections.

IG's main method: auxiliary supervision and shared-pass sampling. The main IG objective is $\|\boldsymbol{f}-\boldsymbol{q}\|_2^2 + \gamma\|\boldsymbol{h}-\boldsymbol{q}\|_2^2$. Both losses update the shared backbone, but the final label remains $\boldsymbol{q}$. Sampling extrapolates the final and intermediate outputs from the same backbone pass [39, Sec. 3.2, Eqs. 3–5]. Thus needing two outputs does not imply two full network passes.

GFT: regression through an unconditional mixture. GFT samples β and conditions the shared model on it; its practical loss mixes the sampling prediction $\boldsymbol{f}_\beta$ with a detached unconditional output evaluated at $\beta = 1$ [2, Eq. 9 and Algorithm 1]:

$$\|\beta\boldsymbol{f}_\beta + (1-\beta)\,\mathrm{sg}(\boldsymbol{u}) - \boldsymbol{q}\|_2^2 = \beta^2\|\boldsymbol{f}_\beta - [\boldsymbol{q} + \alpha(\boldsymbol{q} - \mathrm{sg}(\boldsymbol{u}))]\|_2^2, \quad \alpha = \frac{1-\beta}{\beta}, \quad 0 < \beta \le 1. \tag{B.2}$$

The unconditional evaluation has no gradient, but its shared parameters continue to change. This is a target-extrapolation identity for each update, with a β^2 loss weight; it does not make the reference frozen.

Where SWIFT *differs.* SWIFT uses $\|\boldsymbol{f} - [\boldsymbol{q} + \alpha(\boldsymbol{q}-\boldsymbol{g})]\|_2^2$ with a separately pretrained, frozen, class-matched $\boldsymbol{g}$. For fixed data and noise, the target is independent of the main parameters. There is also an update-level overlap with prediction-gap training: if the strong prediction inside the gap is the current $\boldsymbol{f}$ and the weak $\boldsymbol{g}$ is fixed, then, for $0 \le w < 1$,

$$\nabla_\theta\|\boldsymbol{f} - \boldsymbol{q} - w\,\mathrm{sg}(\boldsymbol{f}-\boldsymbol{g})\|_2^2 = (1-w)\nabla_\theta\|\boldsymbol{f} - (1+\alpha)\boldsymbol{q} + \alpha\boldsymbol{g}\|_2^2, \qquad \alpha = \frac{w}{1-w}. \tag{B.3}$$

Together, these identities locate SWIFT's distinction in its fixed class-matched training reference and the finite-budget analysis in Section 4 and Appendix A. The guide is discarded at sampling; its pretraining cost is included in Appendix D.3. Appendix C specifies the corresponding Gaussian-mixture adaptations rather than treating them as the original training recipes.

C Distance to the optimal denoiser on a Gaussian mixture

This experiment compares what several finite-budget objectives learn about the class-A posterior mean $\mathcal{D}^\star = \mathbb{E}_A[\boldsymbol{x}_0 \mid \boldsymbol{x}_t, \sigma]$. Every arm uses the same main backbone and class-A training data. We score each arm's *implied estimate* of $\mathcal{D}^\star$, since a guided network may have a different regression target.

Comparison metric. For SWIFT, the conditional-mean training target is $\boldsymbol{g}_\alpha = \mathcal{D}^\star + \alpha(\mathcal{D}^\star - \mathcal{D}_{\text{weak}})$ (Lemma A.1). The raw network output therefore targets $\boldsymbol{g}_\alpha$ rather than $\mathcal{D}^\star$. We invert that relation to obtain $\hat{\mathcal{D}} = (\mathcal{D}_\theta + \alpha\,\mathcal{D}_{\text{weak}})/(1+\alpha)$, which equals $\mathcal{D}^\star$ when $\mathcal{D}_\theta = \boldsymbol{g}_\alpha$. Table 3 gives the corresponding conversion for each arm. Conversions use only trained predictions; the known posterior mean is used for scoring. CFG and autoguidance change only sampling, so they are outside this training-objective comparison.

For each method, we report its best setting under the selection rule in Table 4. The MG and GFT arms use frozen references in place of the original evolving predictions; GFT trains a separate model for each setting rather than conditioning a single model on it. IG uses an output after the first hidden layer and the auxiliary-supervision objective, rather than IG's target-correction variant in Appendix B. These are task-specific adaptations of the cited objectives, not full reproductions.

Setup. We draw eight component means and define class A by the first four components. Its clean distribution is $p_A = \frac{1}{4}\sum_{j=1}^{4}\mathcal{N}(\boldsymbol{\mu}_j, s^2\boldsymbol{I}_d)$; the unconditional distribution A+B uses all eight with weight 1/8.

Table 3 Pointwise training losses and evaluated estimates. The expectation over training examples and noise is implicit. Here $\mathcal{D}_{\mathrm{w}}$ is the frozen class-A weak guide, $\mathcal{D}_{\mathrm{c}}$ the frozen class-A baseline, and $\mathcal{D}_{\mathrm{u}}$ the frozen A+B denoiser; sg denotes stop-gradient.

Arm	Pointwise loss	Estimate of $\mathcal{D}^\star$
baseline	$\|\mathcal{D}_\theta - \boldsymbol{x}_0\|_2^2$	$\mathcal{D}_\theta$
SWIFT	$\|\mathcal{D}_\theta - (1+\alpha)\boldsymbol{x}_0 + \alpha\mathcal{D}_{\mathrm{w}}\|_2^2$	$(\mathcal{D}_\theta + \alpha\mathcal{D}_{\mathrm{w}})/(1+\alpha)$
MG [28]	$\|\mathcal{D}_\theta - \boldsymbol{x}_0 - w(\mathcal{D}_{\mathrm{c}} - \mathcal{D}_{\mathrm{u}})\|_2^2$	$\mathcal{D}_\theta - w(\mathcal{D}_{\mathrm{c}} - \mathcal{D}_{\mathrm{u}})$
GFT [2]	$\|\beta\mathcal{D}_\theta + (1-\beta)\,\mathrm{sg}[\mathcal{D}_{\mathrm{u}}] - \boldsymbol{x}_0\|_2^2$	$\beta\mathcal{D}_\theta + (1-\beta)\mathcal{D}_{\mathrm{u}}$
IG (aux. loss) [39]	$\|\mathcal{D}_{\mathrm{f}} - \boldsymbol{x}_0\|_2^2 + \gamma\|\mathcal{D}_{\mathrm{i}} - \boldsymbol{x}_0\|_2^2$	$\mathcal{D}_{\mathrm{f}}$

With $\boldsymbol{x}_t = \boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_d)$, the *class-A* posterior mean corresponding to this setup follows from Lemma A.4:

$$\mathcal{D}_A^\star(\boldsymbol{x}_t, \sigma) = \sum_{j=1}^{4} r_j^A(\boldsymbol{x}_t)\,\frac{s^2\boldsymbol{x}_t + \sigma^2\boldsymbol{\mu}_j}{s^2+\sigma^2},$$
$$r_j^A(\boldsymbol{x}_t) = \frac{\mathcal{N}(\boldsymbol{x}_t; \boldsymbol{\mu}_j, (s^2+\sigma^2)\boldsymbol{I}_d)}{\sum_{\ell=1}^{4}\mathcal{N}(\boldsymbol{x}_t; \boldsymbol{\mu}_\ell, (s^2+\sigma^2)\boldsymbol{I}_d)}. \tag{C.1}$$

Thus $\mathcal{D}_A^\star = \mathcal{D}^\star$ in this experiment. The eight-component distribution supplies the training data for $\mathcal{D}_{\mathrm{u}}$, not the evaluation target. At both $d = 16$ and $d = 8$, the eight means are drawn independently from $\mathcal{N}(\mathbf{0}, 4^2\boldsymbol{I}_d)$ for each seed, with $s = 0.5$. Training samples $\log\sigma \sim \mathcal{U}(-3, 1)$. Every main arm uses a three-hidden-layer width-64 MLP with input $[\boldsymbol{x}_t, \log\sigma]$, paired initialization and class-A training stream, and Adam for 20,000 steps (learning rate 2×10^{-3}, batch size 512). IG additionally trains an intermediate head after the first hidden layer.

The class-A weak guide and the A+B unconditional reference both regress $\boldsymbol{x}_0$ for 4,000 steps with width-16 MLPs, then remain frozen. MG also uses the fixed width-64 class-A baseline as $\mathcal{D}_{\mathrm{c}}$. Thus the main backbone and training budget are matched; the auxiliary predictions used by each arm are specified in Table 3.

Evaluation. For each noise level $\sigma \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ we draw 20,000 fresh samples of class A, noise them, and report the mean Euclidean distance $\frac{1}{N}\sum_i \|\hat{\mathcal{D}}(\boldsymbol{x}_{t,i}) - \mathcal{D}^\star(\boldsymbol{x}_{t,i})\|_2$. Each seed redraws the mixture means and retrains every arm: 40 seeds at d=16 and 5 at d=8. The table reports the mean across these independent mixture-and-training replicates, not the error of an ensemble predictor. These distances evaluate the implied estimates, which combine the main prediction with auxiliary predictions where specified in Table 3; they are not distances of the standalone main network or measurements of sampling cost.

Table 4 Distance to the class-A posterior mean. For each method and dimension, its best setting is selected by the lowest arithmetic mean distance across the five noise levels; the whole row uses that one setting. Means are over independent seeds; all arms share the main backbone. Lower is better; bold marks the best displayed entry at each noise level.

$d = 16$ (40 seeds)					
Method	$\sigma = 0.05$	0.1	0.2	0.5	1.0
baseline	0.3100	0.2600	0.2311	0.2125	0.2272
SWIFT	**0.2566**	**0.2175**	**0.1952**	**0.1865**	**0.1988**
MG	0.3179	0.2685	0.2388	0.2216	0.2358
GFT	0.2617	0.2203	0.1972	0.1904	0.2075
IG	0.3418	0.2885	0.2522	0.2258	0.2312
$d = 8$ (5 seeds)					
Method	$\sigma = 0.05$	0.1	0.2	0.5	1.0
baseline	0.1853	0.1574	0.1378	0.1154	0.1232
SWIFT	**0.1450**	**0.1202**	**0.1120**	**0.1018**	**0.1189**
MG	0.1913	0.1659	0.1491	0.1246	0.1356
GFT	0.1544	0.1264	0.1195	0.1095	0.1257
IG	0.1976	0.1657	0.1468	0.1301	0.1330

Results. Table 4 reports the mean distances. At d=16, SWIFT is lowest at all five noise levels, reducing the baseline distance by 12.2–17.2%. GFT is the closest comparator; its distance exceeds SWIFT's by 0.0020–0.0087. At d=8, the selected SWIFT row is lowest among the displayed methods at all five noise levels. MG and IG have larger errors than the baseline in both displayed comparisons. The small GFT margins and five-seed d=8 comparison should be read as outcomes of this matched setup, not a ranking of the original MG, GFT, and IG implementations.

D Experimental Setup

We specify training (Appendix D.1), evaluation (Appendix D.2), and compute accounting (Appendix D.3). In matched comparisons, SWIFT changes only the training target, leaving the model's architecture, optimizer, schedule, sampler, and evaluator fixed. In the default frozen-guide recipe, the guide receives the same noisy image (or latent), noise level, and class condition as the main denoiser. Its prediction is evaluated without gradients and used only to construct the target in Eq. (5); the original noise distribution, loss weighting, and optimizer update are retained. The frozen guide is not an inference-time component.

D.1 Training Configurations

CIFAR-10. We use the EDM configuration of Karras et al. [10]: a class-conditional DDPM++ model at 32×32 resolution with EDM preconditioning and loss weighting, batch size 512, fp32 training, and a 200,000-kimg horizon (kimg = 10^3 training images). The main denoiser uses channel base 128 (`cb128`). Guide capacity is specified by channel base (`cb16`–`cb128`), with `cb64` denoting the default half-width guide. NCSN++ and ADM-style guide variants use the corresponding channel bases; their full results are reported in Appendix F.

The default guide is a separately trained, class-conditional DDPM++ `cb64` model frozen at 200,000 kimg. Unless a sweep changes it, α is a fixed scalar. The $\alpha = 0$ arm (`D0`) is the ordinary-training reference at the same main-model budget and evaluation endpoint; it is not a new seed-matched baseline trained for every exploratory setting. Appendix F and the accompanying `DATA/` ledger distinguish repeated runs, guide variants, and incomplete or extended runs.

ImageNet-256, DiffusionBench. Our main DiffusionBench model [16] uses a representation-autoencoder tokenizer [38] over frozen DINOv2-B features [22] and a 615M-parameter decoupled diffusion transformer [30].

The breadth study keeps the 615M-parameter training recipe while varying the tokenizer or using a pixel-space model (Appendix E.1). The model is trained class-conditionally with global batch size 1024 for 80 epochs, corresponding to 1251 optimizer steps per epoch, and is evaluated every five epochs.

Unless stated otherwise, the 18 DiffusionBench configurations use the same SWIFT recipe: $\alpha = 0.4$ and a separate guide for the corresponding model, pretrained for half the main model's standard training duration and then frozen. Each half-width guide has about 155M parameters and approximately one quarter of the main denoiser's forward-pass FLOPs. The short-guide ablation freezes the same architecture at an earlier checkpoint. CFG is used only at sampling time and is tuned independently for the baseline and SWIFT.

The ImageNet backends use linear flow matching, $\boldsymbol{x}_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{\epsilon}$ and $\boldsymbol{v} = \boldsymbol{\epsilon} - \boldsymbol{x}_0$, with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, rather than the additive-noise parameterization of Section 2.1. Most DiffusionBench configurations predict velocity. At fixed noisy input and time, velocity is affine in the clean prediction. The implementation constructs the main loss target in velocity space:

$$\boldsymbol{v}_{\text{SWIFT}} = (1+\alpha)\boldsymbol{v} - \alpha\boldsymbol{v}_{\text{g}},$$

This is equivalent to extrapolating the clean-sample target and then converting it to velocity (apart from numerical clamping near a time endpoint). For clean-sample-prediction configurations, including RAEv2, the guide output $\widehat{\boldsymbol{x}}_{\text{g}}$ is first converted to $\boldsymbol{v}_{\text{g}} = (\boldsymbol{x}_t - \widehat{\boldsymbol{x}}_{\text{g}})/\max(t, \varepsilon)$; the model likewise converts the main output before computing its velocity-space loss. The guide is detached, and RAEv2's internal-guidance auxiliary head retains its ordinary target.

The main DiffusionBench sweep is trained on 4×B200 GPUs. Longer-training experiments use the same optimization configuration and are run on 8×H100 GPUs where noted.

ImageNet-256, RAEv2. We use the released 875M-parameter, class-conditional DINOv3-L configuration of Singh et al. [26] (`dinov3l-k7`) together with its native internal-guidance implementation. The `k7` encoder sums layers $11, 13, 15, 17, 19, 21, 23$. The main model predicts clean latents and trains for up to 160 epochs (global batch 1024, bf16, 8×B200 GPUs). The matched baseline keeps the model's gMuon schedule (learning rate 2×10^{-4}, 25-epoch warmup, decay to 2×10^{-5} by epoch 50), latent preprocessing, and EMA decay 0.9995. The SWIFT guide is the model's `GUIDE-w05` weak-model configuration, loaded from a pretrained checkpoint and frozen before main-model training. Unless stated otherwise, we use $\alpha = 0.4$.

The guide components use (width, depth, heads) $= (720, 28, 10)$ and $(1024, 2, 8)$; `w05` denotes approximately half the model's parameters, not half of every hidden width. All other architectural and optimization settings follow the released RAEv2 configuration.

D.2 Sampling and Evaluation Settings

Table 5 summarizes our evaluation settings. Reported metrics use 50,000 generated samples per checkpoint. Published baselines, where used, retain the metrics and evaluation procedure reported by their authors rather than being described as our own re-evaluations.

Table 5 Sampling and evaluation settings used for our runs. Within a re-evaluated pair, the baseline and SWIFT use the same sampler, generated-sample count, and metric implementation. CIFAR-10's 18 Heun steps use 35 denoiser evaluations; inference guidance may add network passes to the other models' stated solver steps.

	CIFAR-10 (EDM)	ImageNet-256 (DiffusionBench)	ImageNet-256 (RAEv2)
resolution / space	32 × 32, pixels	256 × 256, latents or pixels	256 × 256, DINOv3-L latents
sampler	EDM deterministic (Heun) [10]	Euler [16]	Euler [26, App. A, Table 11]
solver steps	18	50	50
guidance	none	CFG, tuned independently	internal guidance, tuned independently
generated samples	50k	50k	50k
reference statistics	CIFAR-10 training set	ImageNet-256 training set	ImageNet-256 training set
metrics	FID	FID, sFID, IS, precision, recall, FD_r, MIND	FID, sFID, IS, precision, recall, FD_r, MIND

Guidance at sampling time. RAEv2's native internal guidance obtains its weak prediction from the first 8 of the backbone's 28 blocks (`base_model_depth=8`), rather than from the separately pretrained SWIFT guide.

For unguided SWIFT evaluation, the training guide is removed entirely. The evaluation script sets `self_guide.k=0` and clears the guide list before checkpoint loading; the training guide is therefore absent from both unguided and natively guided sampling. For guided re-evaluations, CFG or internal-guidance strength is swept separately for each model; we report the best measured FID and compute other metrics on the samples from that setting. A published guided baseline is identified as such, without claiming that we retuned its scale (Figure 7).

Checkpoint selection. DiffusionBench breadth results compare an 80-epoch SWIFT model with an 80-epoch baseline or source-reported baseline; the longer DINOv2-B pair shares an evaluation schedule. RAEv2 has a matched epoch-160 comparison, while its headline 1.15 unguided and 0.99 guided SWIFT values select the epoch-140 checkpoint within the 160-epoch budget (Appendix E.3), not a same-epoch causal comparison.

Metric computation. FID uses the training-set reference statistics of the corresponding dataset [6]. ImageNet-256 sFID, Inception Score [25], precision, recall, FD_r, and MIND use the same generated set as FID. FD_r and MIND cover Inception, ConvNeXt, DINOv2, MAE, and SigLIP; Table 1 averages the five spaces. Re-evaluated pairs share metric implementations and reference features. RAEv2's FID uses the standard ImageNet statistics; its additional representation-space comparisons use our ImageNet training-set feature bank.

D.3 Compute Accounting

We separate main-model training from the one-time guide pretraining, which can serve multiple SWIFT runs.

Wall-clock measurements. On 4×B200 GPUs, 80 DiffusionBench baseline epochs take about 44 hours versus 49 hours for the SWIFT main run; guide pretraining takes another 12 hours once. We use FLOPs for hardware-independent comparison.

Per-step FLOPs. Analytical forward costs are 337.5 GFLOPs per sample for the main denoiser and 87.3 GFLOPs for its half-width guide, a ratio of 0.259. A baseline update uses a main-model forward pass and a backward pass estimated at twice that cost. SWIFT adds one frozen-guide forward pass without guide backpropagation. This raises main-training FLOPs by about 9% per update. A PyTorch FLOP counter agrees with the full-update estimate to within 0.5%.

Guide preparation. For the same number of samples per epoch and the same backward-to-forward ratio, pretraining the guide for half the standard main-model duration costs about 10.4 baseline-equivalent epochs. Charged to a single 80-epoch SWIFT run, the guide and main training together cost about 98 baseline-equivalent epochs, or 1.22 times an 80-epoch baseline. Once the guide is available, each additional main run incurs only the per-update increase of about 9%. These model-only counts exclude the frozen representation encoder, which is shared by both methods.

RAEv2 guide cost. The RAEv2 guide costs about 37% of a main-model forward pass, giving about 12% extra FLOPs per main-training update. Its pretraining costs about 15 baseline-equivalent epochs. Charging that once to a 50-epoch SWIFT model gives a total of about 71 baseline-equivalent epochs, or 0.44 times the cost of a 160-epoch baseline under the same assumptions.

E ImageNet-256: complete results

This appendix provides the complete ImageNet-256 results. We first report the full 19-configuration comparison with and without inference-time guidance (Appendix E.1), followed by the published results used in Figure 1 (Appendix E.2), a detailed analysis of the 875M-parameter RAEv2 model (Appendix E.3), training trajectories (Appendix E.4). Training configurations, evaluation settings, and compute accounting are given in Appendices D.1 to D.3.

E.1 All configurations

The 19 configurations consist of 18 DiffusionBench configurations (615M parameters, 80 epochs; six representation autoencoders, five VAEs, four end-to-end-tuned autoencoders, and three pixel-space models) together with the larger RAEv2 model. Across all configurations, SWIFT uses the same default coefficient, α=0.4, with a smaller pretrained guide frozen before main-model training; no coefficient is tuned per configuration. Each baseline is either a reference run trained with the same schedule or a released checkpoint. Our DiffusionBench runs and available matched baselines are evaluated with 50k samples under one pipeline; published values remain as reported. RAEv2 uses its native evaluator.

Without guidance. SWIFT lowers FID in all 18 DiffusionBench configurations (Table 6), by 16.3–49.4% (median 31.3%), and improves RAEv2 from 1.56 to 1.15. Across all 19 configurations, 269 of the 285 reported metric comparisons favor SWIFT, 7 are ties at the displayed precision, and 9 favor the baseline. The exceptions are limited to four precision entries, two Inception Scores, one recall entry, and one ConvNeXt-based FD_r/MIND pair.

With guidance. Guidance is tuned separately where both checkpoints are available; source-reported guided baselines retain their reported settings (Table 7). SWIFT has the lower guided FID in all 19 configurations: by 0.10–1.31 (6.7–33.0%, median 21.9%) across DiffusionBench with CFG, and from 1.04 to 0.99 on RAEv2 with its native internal guidance. Of the 270 DiffusionBench metric comparisons, 252 favor SWIFT, 14 favor the baseline, and 4 are ties at the reported precision. Figure 7 illustrates six representative DiffusionBench cases and also shows why the guidance scale must be tuned separately for the two models.

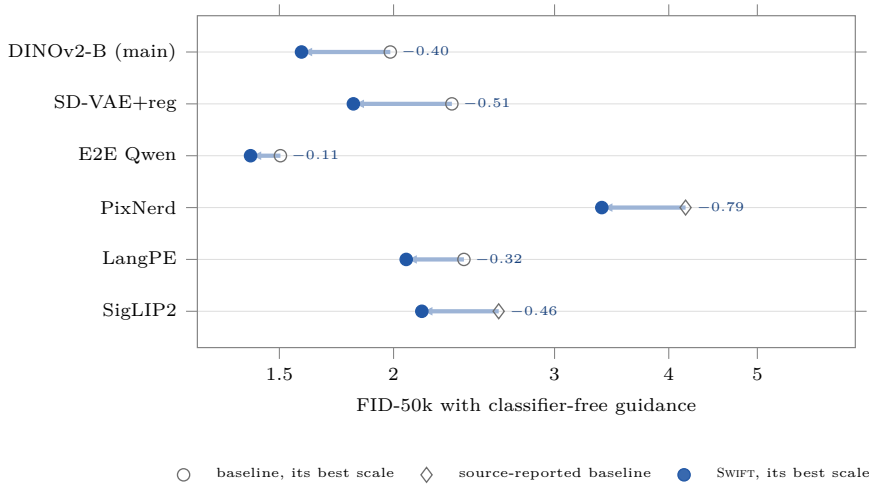

Figure 7 Guided FID at each model's best available CFG scale for six DiffusionBench configurations from Table 7. Open circles denote matched baselines whose guidance scale is swept independently; open diamonds denote source-reported guided baselines. Filled dots show SWIFT at its own best scale, and annotations give the FID change. The preferred scale can shift substantially (e.g., 2.2→1.8 for SD-VAE+reg and 2.4→1.8 for PixNerd), motivating separate tuning on the two sides.

Table 6 All 19 ImageNet-256 configurations without guidance. SWIFT uses the same default coefficient (α=0.4) with a smaller pretrained guide frozen before main-model training, without per-configuration coefficient tuning. FD_r and MIND are reported in five feature spaces; the lower FID of each pair is bold.

						$FD_r \downarrow$					MIND $\downarrow$				
Configuration (baseline)	FID↓	sFID↓	IS↑	Prec.↑	Rec.↑	Inception	ConvNeXt	DINOv2	MAE	SigLIP	Inception	ConvNeXt	DINOv2	MAE	SigLIP
Representation autoencoders (RAE); 615M, 80 epochs															
DINOv2-B	2.09	4.48	210.6	0.83	0.57	1.32	2.34	3.19	6.25	7.57	2.45	70.70	26.72	0.44	6.41
+ SWIFT	**1.75**	4.26	202.8	0.81	0.61	1.09	2.43	2.87	5.18	6.89	1.55	76.44	23.60	0.35	5.76
DINOv2-B+reg	2.07	4.55	208.2	0.82	0.58	1.29	2.42	3.23	6.33	7.67	2.27	77.41	27.21	0.44	6.21
+ SWIFT	**1.71**	4.18	205.2	0.81	0.61	1.09	2.40	2.90	5.29	6.75	1.56	74.47	23.36	0.35	5.79
LangPE	2.73	4.43	183.2	0.80	0.60	1.62	3.41	6.57	9.00	10.10	2.55	135.04	63.77	0.69	8.79
+ SWIFT	**2.27**	4.01	184.6	0.79	0.61	1.36	3.32	5.71	7.60	9.29	1.77	126.68	55.29	0.56	8.40
PE	3.11	6.15	205.1	0.81	0.56	1.88	3.12	6.12	11.01	9.83	3.20	97.46	58.28	0.88	8.07
+ SWIFT	**2.34**	5.02	209.6	0.81	0.60	1.43	2.71	4.76	9.18	7.74	2.47	86.73	44.90	0.70	6.39
SigLIP2	3.47	7.02	180.5	0.79	0.58	2.09	4.13	8.61	11.01	11.66	2.45	158.79	86.24	0.90	10.18
+ SWIFT	**2.60**	5.39	188.3	0.79	0.61	1.59	3.55	7.00	9.09	9.60	1.89	132.62	70.05	0.72	8.35
SpatialPE	3.60	5.20	160.1	0.78	0.60	2.20	4.21	7.19	6.95	10.73	2.23	210.99	68.81	0.54	9.14
+ SWIFT	**2.92**	4.47	165.8	0.78	0.62	1.78	3.70	6.07	5.60	9.27	1.60	174.36	57.57	0.43	8.02
VAEs; 615M, 80 epochs															
SD-VAE (EMA)+reg	5.49	5.52	161.3	0.71	0.66	3.21	5.26	11.51	9.67	25.77	3.52	257.98	148.15	0.87	28.66
+ SWIFT	**3.77**	5.06	179.1	0.73	0.66	2.19	3.85	8.58	7.56	20.01	2.00	157.58	107.03	0.68	21.85
SD-VAE (MSE)	10.06	5.79	110.9	0.67	0.65	5.92	10.48	17.97	12.24	38.65	9.19	686.07	244.94	1.09	44.55
+ SWIFT	**6.51**	5.26	137.1	0.70	0.66	3.82	7.02	12.97	9.77	30.52	4.94	388.49	170.63	0.87	35.39
SD-VAE (EMA)	10.21	5.56	112.4	0.67	0.66	6.01	10.01	18.12	11.12	35.26	9.69	685.61	244.65	0.99	38.47
+ SWIFT	**6.73**	5.05	138.9	0.70	0.67	3.87	6.65	13.03	8.91	27.75	5.21	388.35	171.18	0.79	30.49
Qwen-VAE	10.69	8.46	109.1	0.66	0.64	6.43	13.29	19.59	13.41	41.22	9.09	806.84	266.43	1.31	43.80
+ SWIFT	**7.18**	7.68	134.8	0.69	0.65	4.13	8.88	14.24	10.72	32.64	4.91	454.12	186.14	1.05	34.72
SDXL-VAE	12.84	7.52	104.1	0.65	0.64	7.48	12.63	21.45	13.58	40.39	12.90	862.20	299.17	1.22	43.99
+ SWIFT	**8.38**	6.83	128.7	0.68	0.65	4.84	8.39	15.50	10.91	31.80	6.98	486.07	209.20	0.97	34.91
End-to-end tuned autoencoders (E2E); 615M, 80 epochs															
E2E VA-VAE	4.14	4.57	148.9	0.74	0.65	2.55	5.80	8.58	6.34	16.22	2.13	270.48	103.26	0.50	15.36
+ SWIFT	**3.13**	4.39	164.2	0.74	0.67	1.96	4.62	6.82	5.21	12.99	1.25	188.11	80.39	0.41	12.24
E2E Qwen-VAE	4.94	5.23	138.9	0.72	0.66	3.07	6.75	9.60	6.48	19.94	2.82	339.60	114.11	0.53	18.94
+ SWIFT	**3.23**	4.89	164.1	0.73	0.66	2.08	4.66	6.41	4.74	14.48	1.34	190.38	72.89	0.37	13.18
E2E SD3.5-VAE	5.26	5.18	141.1	0.71	0.65	3.34	5.03	9.14	7.13	18.44	3.17	260.37	107.42	0.55	17.58
+ SWIFT	**3.61**	4.88	160.9	0.72	0.67	2.37	3.66	6.73	5.50	13.99	1.71	160.89	74.95	0.41	13.00
E2E FLUX-VAE	6.20	5.13	134.9	0.70	0.66	3.76	6.84	10.89	6.63	21.30	4.07	372.55	127.66	0.53	20.24
+ SWIFT	**4.27**	4.78	153.4	0.71	0.67	2.61	4.87	7.80	5.00	15.54	2.21	220.74	87.92	0.39	14.49
Pixel space; 615M, 80 epochs															
PixelGen	12.10	6.15	104.2	0.66	0.61	7.00	13.92	17.92	18.96	34.61	9.04	771.89	222.22	1.68	32.40
+ SWIFT	**8.11**	5.46	126.3	0.68	0.62	4.72	9.80	13.72	15.89	27.86	5.03	481.69	166.83	1.39	26.36
PixNerd	20.37	6.14	65.9	0.61	0.63	12.00	20.71	24.44	18.69	46.76	22.30	1513.80	335.52	1.62	47.63
+ SWIFT	**10.31**	5.38	93.3	0.70	0.61	6.14	12.20	14.33	13.80	29.92	8.50	776.86	172.29	1.23	28.80
JiT	22.36	6.98	63.3	0.58	0.63	13.20	23.38	28.66	22.69	54.69	25.50	1690.32	422.58	2.04	56.31
+ SWIFT	**12.31**	6.15	98.4	0.64	0.63	7.27	13.52	18.50	18.20	38.32	11.10	805.46	254.28	1.65	39.33
RAEv2, 875M, up to 160 epochs															
DINOv3-L	1.56	4.10	233.3	0.82	0.65	1.01	2.24	2.63	4.84	6.32	1.44	70.17	22.46	0.35	5.61
+ SWIFT	**1.15**	3.60	242.2	0.81	0.67	0.81	2.05	2.24	4.17	5.41	1.11	65.39	18.56	0.29	4.82

Table 7 All 19 ImageNet-256 configurations with guidance. Inference-time guidance is tuned separately for re-evaluated pairs; source-reported guided baselines retain their published settings. The training recipe is otherwise the same as in Table 6. FD_r and MIND are reported in five feature spaces; the lower FID of each pair is bold.

						$FD_r \downarrow$					MIND $\downarrow$				
Configuration (baseline)	FID↓	sFID↓	IS↑	Prec.↑	Rec.↑	Inception	ConvNeXt	DINOv2	MAE	SigLIP	Inception	ConvNeXt	DINOv2	MAE	SigLIP
Representation autoencoders (RAE); 615M, 80 epochs															
DINOv2-B	1.98	4.50	223.1	0.81	0.59	1.26	2.21	3.40	6.12	7.84	1.91	66.75	27.95	0.43	6.52
+ SWIFT	**1.59**	4.30	221.6	0.80	0.62	1.00	2.27	2.78	5.14	6.65	1.15	66.61	23.23	0.33	5.70
DINOv2-B+reg	1.84	4.55	236.2	0.82	0.58	1.16	2.16	3.24	6.37	7.96	1.91	64.08	27.03	0.43	6.01
+ SWIFT	**1.52**	4.18	232.9	0.81	0.61	0.98	2.10	2.73	5.31	6.57	1.31	62.96	22.34	0.35	5.92
LangPE	2.39	4.42	201.6	0.80	0.61	1.44	2.95	6.08	9.00	9.95	2.16	104.11	62.58	0.71	8.85
+ SWIFT	**2.06**	4.02	199.0	0.79	0.62	1.26	2.84	5.44	7.54	9.44	1.40	103.39	53.36	0.55	8.82
PE	2.84	5.80	221.5	0.81	0.57	1.72	2.78	5.87	10.68	9.89	2.82	89.34	56.48	0.87	7.88
+ SWIFT	**2.14**	4.73	226.4	0.80	0.62	1.32	2.44	4.61	9.05	7.78	2.20	78.90	44.01	0.71	6.44
SigLIP2	2.61	6.90	222.9	0.78	0.59	1.62	2.97	7.31	10.76	11.33	1.85	97.56	75.42	0.87	9.73
+ SWIFT	**2.15**	5.32	219.4	0.79	0.62	1.32	2.82	6.29	8.76	9.68	1.33	92.86	66.35	0.69	8.72
SpatialPE	1.86	4.50	247.1	0.79	0.61	1.18	1.82	4.76	6.58	8.29	1.44	54.32	46.04	0.51	6.84
+ SWIFT	**1.51**	3.87	255.9	0.79	0.63	0.97	1.60	3.87	5.44	7.71	1.04	44.44	36.64	0.40	6.52
VAEs; 615M, 80 epochs															
SD-VAE (EMA)+reg	2.32	4.70	271.5	0.80	0.60	1.32	1.73	7.42	7.89	18.58	1.90	53.68	85.97	0.71	20.32
+ SWIFT	**1.81**	4.42	272.9	0.80	0.61	0.99	1.46	5.82	6.19	15.20	1.64	46.10	65.92	0.55	16.52
SD-VAE (MSE)	2.56	5.50	259.7	0.70	0.65	1.43	2.14	7.60	7.93	21.85	3.09	55.50	87.48	0.69	24.63
+ SWIFT	**1.86**	4.99	268.4	0.73	0.66	0.91	1.42	5.33	6.23	17.13	1.62	30.65	59.76	0.55	19.54
SD-VAE (EMA)	2.43	5.40	259.6	0.69	0.65	1.39	1.94	7.70	6.97	19.19	2.87	49.03	88.22	0.62	20.47
+ SWIFT	**1.90**	5.08	260.9	0.69	0.66	1.07	1.60	5.99	5.40	15.37	2.52	42.61	67.34	0.49	16.57
Qwen-VAE	3.01	7.70	238.9	0.68	0.65	1.83	4.59	9.84	9.27	25.20	2.75	160.49	118.34	0.91	25.93
+ SWIFT	**2.32**	6.99	250.8	0.71	0.66	1.19	3.07	7.06	7.41	20.16	1.48	90.34	82.45	0.70	20.68
SDXL-VAE	3.07	6.90	256.0	0.67	0.65	1.71	2.76	9.28	8.41	21.84	3.32	70.24	108.68	0.74	22.53
+ SWIFT	**2.21**	6.27	268.8	0.70	0.66	1.09	1.84	6.78	6.61	17.01	1.79	39.10	76.95	0.61	17.94
End-to-end tuned autoencoders (E2E); 615M, 80 epochs															
E2E VA-VAE	1.65	4.50	275.4	0.74	0.65	1.09	1.96	4.61	4.82	9.82	2.16	62.56	51.43	0.36	9.01
+ SWIFT	**1.25**	4.32	289.2	0.73	0.67	0.81	1.61	3.53	3.80	7.91	1.25	43.97	40.60	0.31	7.24
E2E Qwen-VAE	1.50	4.83	263.3	0.80	0.62	1.05	2.29	4.58	4.61	11.10	1.45	61.43	48.55	0.37	10.20
+ SWIFT	**1.40**	4.59	296.5	0.80	0.63	0.98	1.75	3.14	3.31	7.49	1.81	78.14	30.31	0.26	6.50
E2E SD3.5-VAE	1.62	5.00	265.4	0.72	0.66	1.14	1.27	4.48	5.44	11.20	1.09	42.67	47.19	0.42	10.30
+ SWIFT	**1.31**	4.81	278.7	0.71	0.68	0.88	1.03	3.47	4.47	8.92	0.65	31.27	36.05	0.33	8.13
E2E FLUX-VAE	1.67	5.00	266.3	0.70	0.66	1.04	1.81	5.17	4.57	11.60	1.16	51.16	54.27	0.36	10.39
+ SWIFT	**1.25**	4.66	279.6	0.71	0.67	0.74	1.33	3.56	3.48	8.84	0.63	32.49	36.09	0.27	7.90
Pixel space; 615M, 80 epochs															
PixelGen	3.97	5.50	247.4	0.67	0.61	2.27	4.29	7.85	13.09	17.80	3.99	137.50	86.22	1.12	15.81
+ SWIFT	**2.66**	4.88	259.8	0.69	0.63	1.53	3.07	6.07	11.06	14.28	2.20	86.24	65.50	0.93	12.93
PixNerd	4.17	5.50	213.8	0.65	0.63	2.45	4.00	8.46	11.70	20.87	4.21	103.10	86.18	0.98	18.85
+ SWIFT	**3.38**	4.95	235.2	0.68	0.62	1.96	3.28	6.82	9.54	17.16	3.38	85.25	70.80	0.79	15.36
JiT	4.08	6.00	231.2	0.60	0.63	2.38	4.60	9.65	13.59	22.44	3.93	147.18	114.34	1.23	21.53
+ SWIFT	**2.86**	5.29	277.4	0.66	0.64	1.31	2.63	6.21	10.91	15.81	1.73	71.94	68.08	0.99	14.87
RAEv2, 875M, up to 160 epochs															
DINOv3-L	1.04	4.00	266.0	0.82	0.61	0.87	1.66	2.67	5.21	6.16	1.34	49.36	22.47	0.34	4.84
+ SWIFT	**0.99**	3.80	264.9	0.81	0.63	0.82	1.59	2.41	4.83	5.69	1.25	46.11	20.14	0.30	4.46

E.2 Published results in Figure 1

Table 8 lists the published results plotted in the left panel of Figure 1. Every number is the one reported by the authors for their best configuration at 256×256; we did not re-evaluate these models.

Table 8 Published ImageNet-256 results shown in Figure 1 (left). FID-50K values are taken directly from the corresponding papers. Evaluation procedures differ between papers: some sample 50 images per class, others sample class labels at random, which can change FID by about 0.1 [38, Table 14]. Guidance: CFG, classifier-free guidance; int., with a guidance interval; AG, autoguidance; IG, internal guidance [39]; REPA guid., the REPA guidance of Singh et al. [26]. *Without CFG, but still with internal guidance. Our two RAEv2 rows use the settings of Appendix E.3.

Model	Type	Base FID	Guided FID	Guidance
DiT-XL/2 [24]	latent diffusion	9.62	2.27	CFG
SiT-XL/2 [20]	latent diffusion	8.30	2.06	CFG
MDT-XL/2 [5]	latent diffusion	6.23	1.79	CFG
DDT-XL/2 [30]	latent diffusion	6.27	1.26	CFG, int.
REPA [35]	latent diffusion	5.90	1.42	CFG, int.
MAR-H [17]	autoregressive	2.35	1.55	CFG
SFD-XXL [23]	latent diffusion	2.38	1.04	AG
LightningDiT [33]	latent diffusion	2.17	1.35	CFG, int.
REG [31]	latent diffusion	1.80	1.36	CFG, int.
REPA-E [15]	latent diffusion	1.69	1.12	CFG, int.
RAEv2 [26]	latent diffusion	1.65	1.06	REPA guid.
RAE [38]	latent diffusion	1.51	1.13	AG
PAE [37]	latent diffusion	1.43	1.03	CFG
BAR-L [34]	autoregressive	1.42	0.99	CFG
LightningDiT+IG [39]	latent diffusion	1.34*	1.19	IG, CFG, int.
GAE [19]	latent diffusion	1.31	1.13	CFG, int.
RAEv2, our baseline	latent diffusion	1.56	1.04	Appendix E.3
SWIFT on RAEv2 (ours)	latent diffusion	**1.15**	**0.99**	Appendix E.3

E.3 RAEv2

RAEv2 provides a second, stronger backbone: an 875M-parameter model on DINOv3-L latents with native internal guidance. SWIFT uses the default α=0.4 recipe and a pretrained `GUIDE-w05` guide frozen before main training (Appendix D.1). The canonical RAEv2 results are those reported in Tables 6 and 7: 1.56 → 1.15 without guidance and 1.04 → 0.99 with independently tuned internal guidance.

Training trajectory. To examine when the gain appears, we additionally track SWIFT at 17 checkpoints and the α=0 baseline at epochs 20, 40, 80, 120, and 160 (Figure 8). The α=0.4 run is already below the released epoch-80 checkpoint by epoch 40 and remains on a substantially lower trajectory thereafter. Its best observed unguided checkpoint occurs at epoch 140 and yields the 1.15 value reported in Table 6. The denser checkpoint trace is used only to study training dynamics; the headline values remain those in the two complete-result tables.

Interaction with internal guidance. RAEv2 uses

$$D_s = D + (s-1)(D - D_{\text{REPA}}),$$

so $s = 1$ disables guidance. The preferred guidance strength shifts downward after SWIFT training, while the best guided result improves from 1.04 to 0.99 (Table 7). This is consistent with the main denoiser absorbing part of the weak-to-strong correction during training, while inference-time guidance remains complementary.

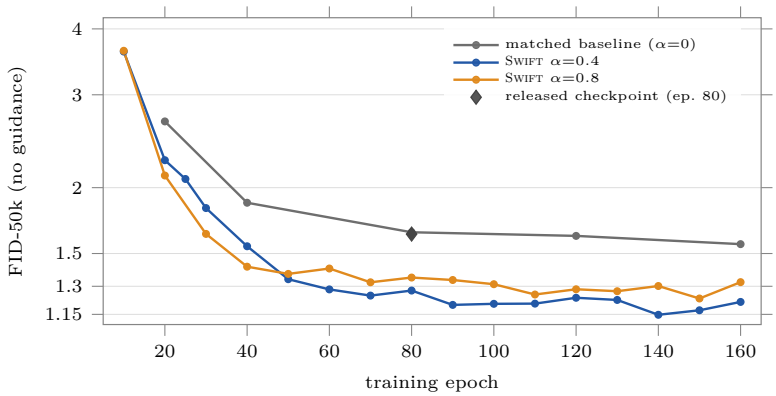

Figure 8 RAEv2 unguided training trajectories. The dense SWIFT traces are used to study training dynamics; the canonical RAEv2 results are reported in Tables 6 and 7. The diamond marks the released epoch-80 checkpoint.

E.4 Training trajectories and the long schedule

Across configurations. For the 12 DiffusionBench configurations whose baseline we also evaluated during training, SWIFT is ahead at every evaluation from epoch 40 on. Figure 9 shows four of them, with baselines spanning roughly an order of magnitude in FID.

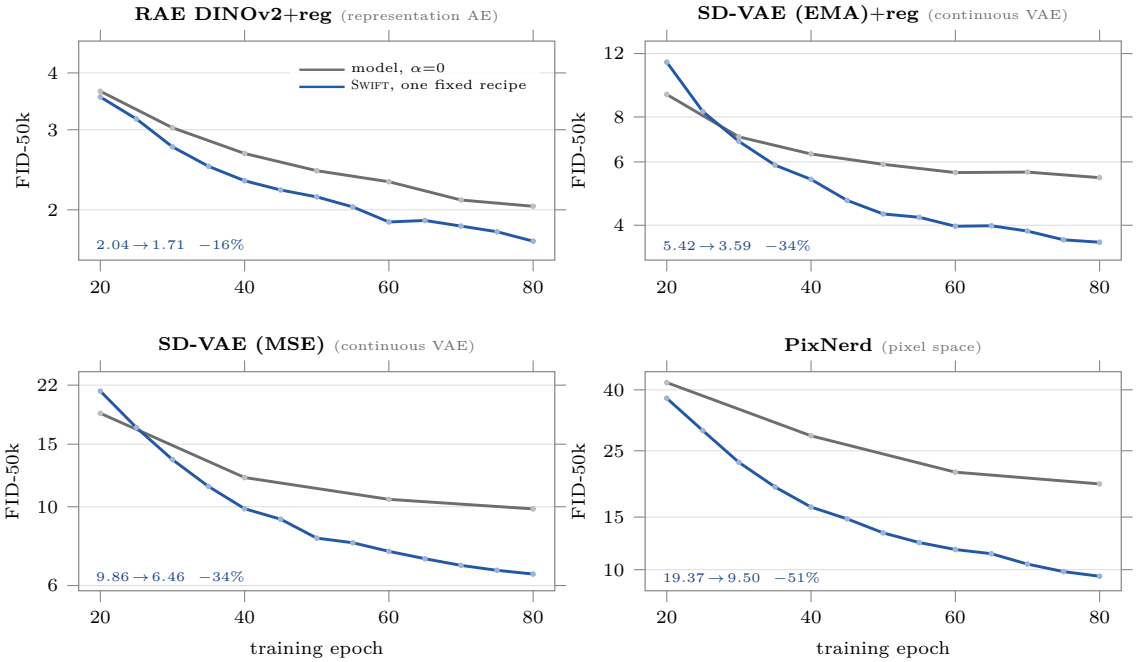

Figure 9 The same fixed SWIFT recipe improves four different ImageNet-256 training trajectories. Each panel shows a different DiffusionBench stage-2 model and representation. Grey is the matched α=0 baseline and blue is SWIFT with the same fixed recipe (α=0.4, half-width guide pretrained for half the standard main-model duration and then frozen), with no per-model tuning. SWIFT is ahead by epoch 40 in all four cases and remains ahead through the end of training, across baselines spanning roughly an order of magnitude in FID. The y-axis is logarithmic; faint markers denote individual FID evaluations. The trajectory baselines reproduce the corresponding DiffusionBench endpoints within 0.6–6%; the comparisons in Table 6 instead re-evaluate both sides with 50k samples.

Guide maturity on DINOv2-B. In the matched 80-epoch ablation, the default guide is pretrained for half the standard main-model duration. Its α=0.4 and 0.8 arms lag the baseline at epoch 20 (by 0.11 and 0.05 FID), but lead from epoch 40 onward (by 0.28–0.48). An earlier guide with α=0.4 leads throughout (by 0.47–0.59 FID).

Persistence through 240 epochs. In a separate, hardware-matched DINOv2-B pair using the fixed α=0.4, default frozen-guide recipe, we train both arms to 240 epochs. Across 36 matched evaluations from epoch 20 onward, SWIFT has lower FID at every checkpoint. The gap remains 0.26 FID at epoch 240, so the gain does not vanish on this longer schedule. Table 9 reports FID values every 20 epochs, and Figure 10 shows the corresponding training trajectories. A separate α=0 hardware replicate is an additional reference only and is excluded from the differences.

Table 9 DINOv2-B over 240 matched training epochs. Unguided FID from the training-time evaluator, sampled every 20 epochs. The two blocks list epochs in order; Δ is SWIFT minus baseline, computed before rounding the FIDs displayed here. Figure 10 plots the same 20-epoch checkpoints.

Epoch	Baseline	SWIFT	Δ	Epoch	Baseline	SWIFT	Δ
20	3.63	**3.52**	−0.12	140	1.81	**1.51**	−0.30
40	2.67	**2.32**	−0.36	160	1.80	**1.46**	−0.34
60	2.36	**2.00**	−0.36	180	1.75	**1.49**	−0.26
80	2.13	**1.77**	−0.37	200	1.72	**1.44**	−0.28
100	1.99	**1.60**	−0.39	220	1.67	**1.41**	−0.26
120	1.91	**1.50**	−0.41	240	1.63	**1.38**	−0.26

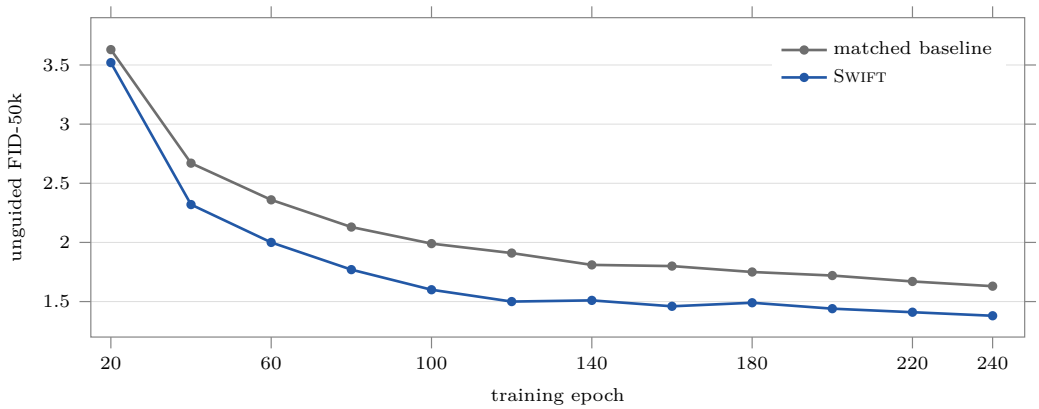

Figure 10 DINOv2-B training trajectories through epoch 240. The matched unguided FID curves are plotted every 20 epochs, using the rounded values in Table 9. SWIFT remains below the baseline throughout the displayed schedule.

F Additional CIFAR-10 experiments

Experimental setting. Beyond the coefficient, guide-width, and guide-maturity ablations in Section 5.4, we tested the additional methods below. We use class-conditional CIFAR-10 at 32×32, a DDPM++ student with channel base 128, batch size 512, and 200k kimg of training. Unless varied by the method, the guide is a separately trained, frozen DDPM++ model with channel base 64 at 200k. Evaluation uses FID-50k with the 18-step EDM sampler.

Table 10 Additional methods and their best FIDs. Each entry is the lowest FID at the common 200k endpoint across all completed settings and runs of that method, rather than a run average. Online adaptive rules are preliminary.

Method	Best FID
Single-stage guide-capacity switch	1.501
Time profile: late peak	1.507
Combined time/noise coefficients	1.549
Recorded guide ahead of student training age	1.557
Time profile: late hold	1.559
Multi-stage guide-capacity schedule	1.561
Terminal-α changes (profiles or shared-prefix tails)	1.562
Normalized noise-dependent α tilts	1.568
NCSN++ guides (width 64/128)	1.570
Negative-α warm starts	1.582
Recorded same-age guide	1.602
Co-trained guide	1.618
Time profile: early/middle/tail	1.619
Recorded guide behind student training age	1.621
Noise-dependent α ramps	1.622
Random $\alpha \sim U[0, 1]$	1.625
Time profile: piecewise	1.630
ADM guides (width 96)	1.639
Time profile: middle only	1.827
Online adaptive α rules (preliminary)	2.416

Guide variants. We replaced the DDPM++ guide with NCSN++ or ADM, co-trained the guide alongside the student, or replayed separately trained guide checkpoints. The recorded checkpoints either matched the student's training age or were ahead/behind it; ahead variants require pre-trained future checkpoints. Capacity schedules switched guide widths during training, either once or through several stages, including $16 \to 32 \to 64$ and a further width-96 stage.

Coefficient schedules. We varied α over training using early/middle/tail and piecewise profiles, a middle-only pulse, or late-onset peak/hold profiles. We also tested a short negative-α prefix and different terminal coefficients. Noise-dependent variants used ramps or normalized tilts over noise level; other trials sampled α uniformly from $[0, 1]$ or adjusted it online using short loss probes.

Combined methods. We multiplied a training-time coefficient profile by a noise-dependent factor, allowing the extrapolation strength to vary jointly with training progress and noise level. This tests whether the two adjustments can provide complementary improvements. Full settings and all 156 run labels (152 scored), including extended-budget and exploratory runs, remain in the accompanying `DATA` files.

G Additional samples

Figures 11 to 14 show samples from SWIFT on RAEv2/DINOv3-L, the model with guided FID 0.99 in Table 7, for 32 ImageNet classes. As in Figure 3, samples are generated with retuned internal guidance and a 50-step Euler ODE sampler. Each block shows 28 samples of one class; the class index and name are given below the block.

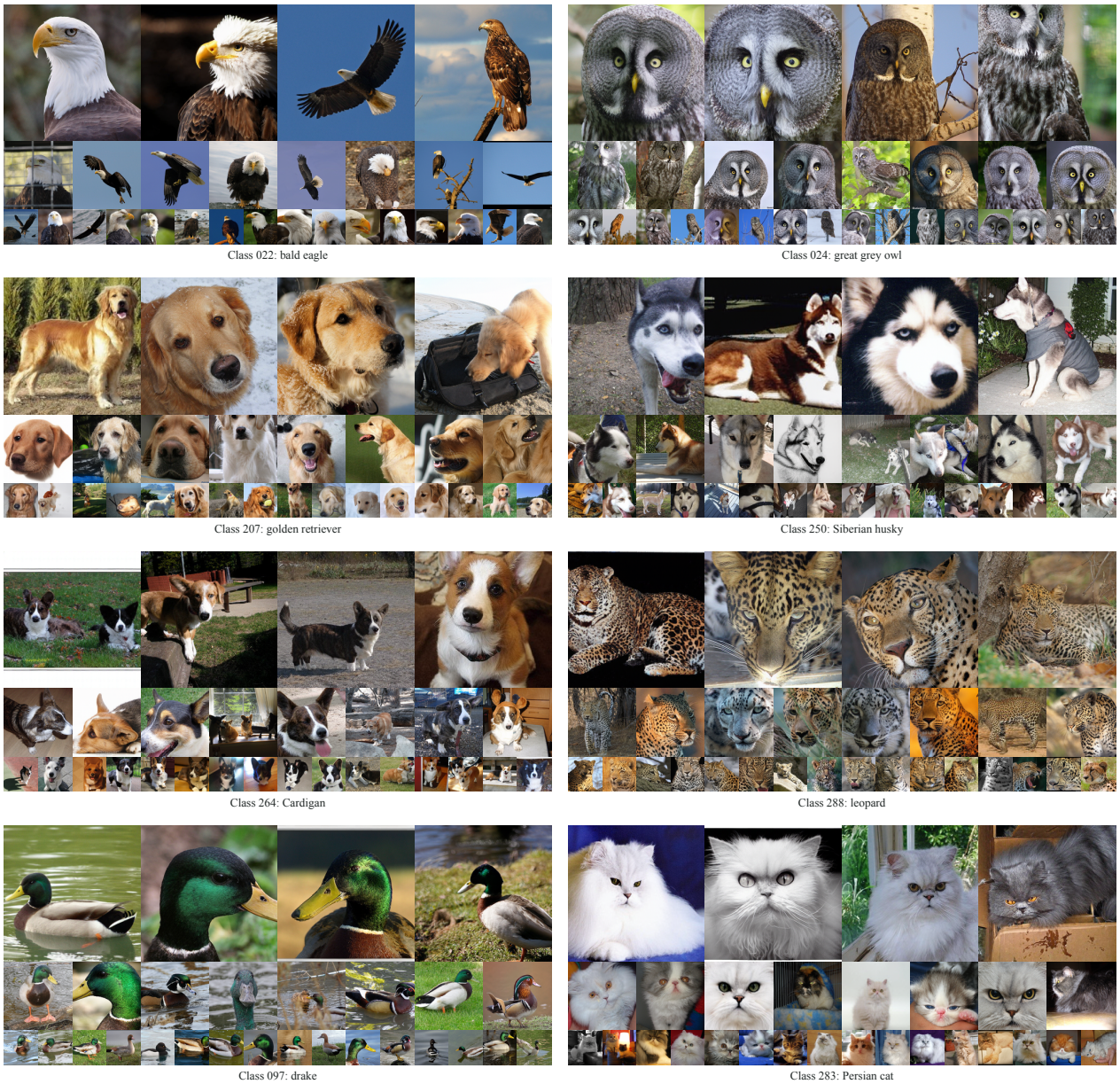

Figure 11 Samples from SWIFT on RAEv2/DINOv3-L: birds and mammals. Internal guidance, 50-step Euler ODE sampler; 28 samples per class.

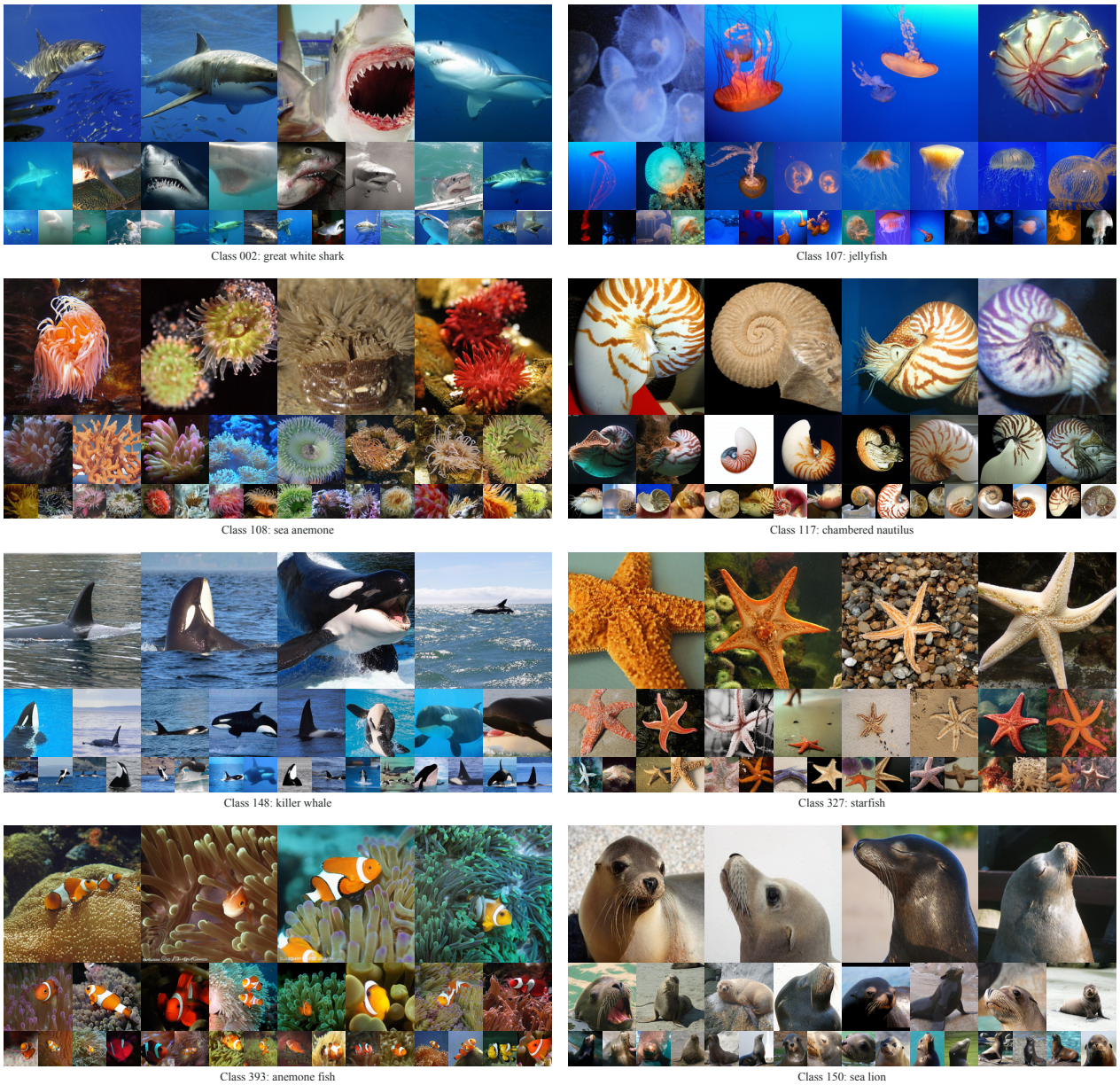

Figure 12 Samples from SWIFT on RAEv2/DINOv3-L: marine life. Internal guidance, 50-step Euler ODE sampler; 28 samples per class.

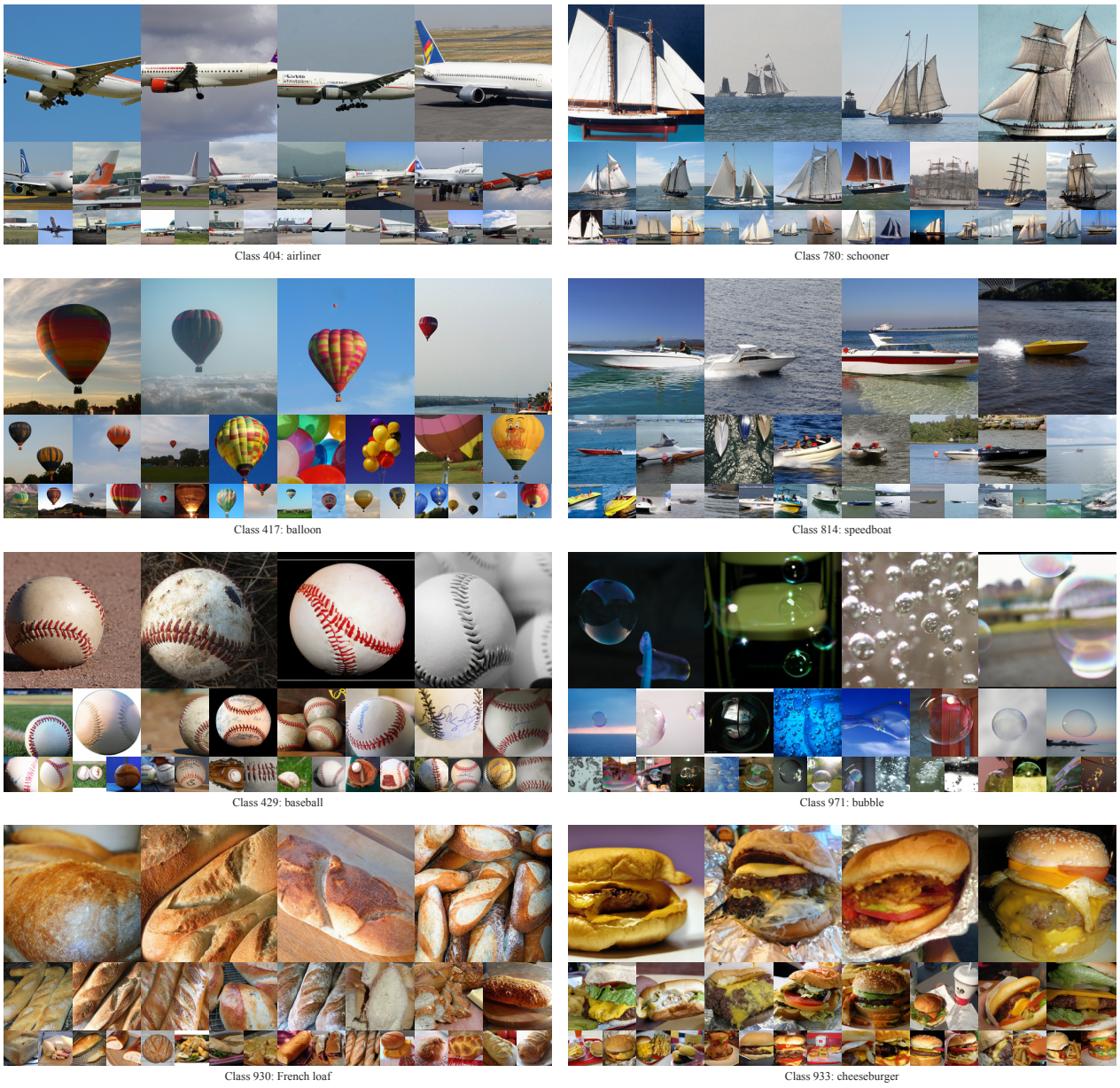

Figure 13 Samples from SWIFT on RAEv2/DINOv3-L: vehicles, objects, and bread. Internal guidance, 50-step Euler ODE sampler; 28 samples per class.

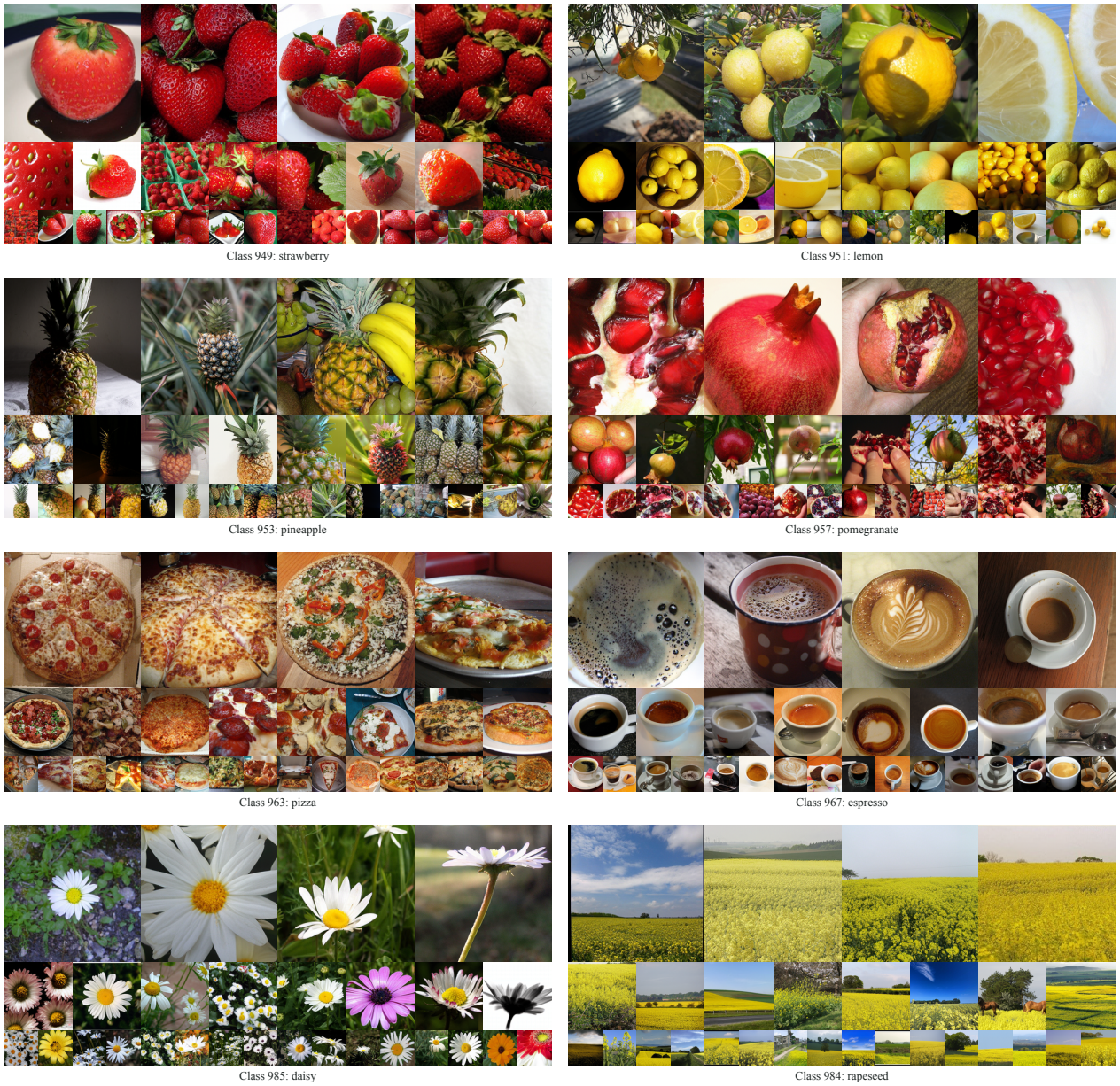

Figure 14 Samples from SWIFT on RAEv2/DINOv3-L: fruit, food, and plants. Internal guidance, 50-step Euler ODE sampler; 28 samples per class.